

CS336 作业 5（对齐）：推理强化学习

版本：26.0.0

CS336 教学团队

2026 年春季

翻译说明：本文件为原 PDF 的中文翻译版。代码、函数名、文件路径、命令、模型名、数据集名、数学符号和公式尽量保留原样；自然语言说明与题目要求翻译为中文。公式编号与原文对应。实际运行代码或复制 prompt 时，仍建议以课程仓库中的原文件为准。

1 作业概览

本作业将让你获得一些亲手训练语言模型进行推理并解决下游任务的实践经验。

你将实现的内容

1. 零样本（zero-shot）、少样本（few-shot）和思维链（chain-of-thought）提示。
2. Group Relative Policy Optimization（GRPO）：一种利用外部 reward 改善模型性能的强化学习算法。
3. 多种 policy gradient 估计变体，用于探索方差降低（variance reduction）和 importance weight clipping 策略。

你将运行的实验

1. 测量 OLMo-2-0425-1B 在 GSM8K 上的提示性能。
2. 在 OLMo-2-0425-1B 上运行 on-policy GRPO，提高其 GSM8K 性能。
3. 运行多种 RL 变体，包括 RFT、Dr. GRPO 和 MaxRL，探索 RL 中不同算法设计选择。
4. 运行 off-policy GRPO，以加快训练并探索不同的 clipping 策略。

代码结构

所有作业代码及本讲义都位于 GitHub：

`github.com/stanford-cs336/assignment5-alignment`

请 `git clone` 仓库。如有更新，课程团队会通知你，你可以使用 `git pull` 获取最新版。

1. `cs336_alignment/*`：你将在这里编写作业 5 的代码。除下面介绍的 starter code 外，这里没有现成实现，因此你基本可以从零组织自己的代码。
2. `cs336_alignment/vllm_utils.py`：包含启动 vLLM server、生成文本以及同步权重的代码。
3. `cs336_alignment/drgrpo_grader.py`：包含用于给数学题模型输出评分的代码。
4. `cs336_alignment/prompts/*`：提供了若干 prompt 文本文件。
5. `tests/*.py`：包含必做 GRPO 测试和可选 alignment/safety supplement 测试。

必做测试 `tests/test_grpo.py` 会调用 `tests/adapters.py` 中定义的 hooks。你需要实现 adapters，把自己的代码接到测试上。自行编写更多测试或修改测试代码可以帮助调试，但你的实现应能通过原始提供的测试套件。

6. `README.md`：包含基本环境设置说明。

提交方式

你需要在 Gradescope 提交：

- `writeup.pdf`：回答所有书面问题。请排版后提交。

- `code.zip`: 包含你编写的全部代码。

运行 `test_and_make_submission.sh` 脚本生成 `code.zip`。

2 引言

2.1 背景

在本课程前四次作业中，我们学习了如何预训练一个 base model。现在我们准备进入**后训练（post-training）**：当我们已经拥有 base model 后，如何把它变成一个真正有用、能够完成下游任务的工具？

后训练的一部分是**对齐（alignment）**。预训练使用的数据混合和训练目标，会让模型获得广泛的知识 and 行为模式。但当我们要求模型完成任务时，我们往往希望它表现出更具体的行为，例如成为一个“有帮助且无害”的助手。把一个通用 base model 转化为专门的聊天模型，这一过程通常称为 **instruction tuning** 或 **alignment**。这些技术会在可选的作业 5 补充材料中介绍。

后训练的另一部分是**强化学习（reinforcement learning, RL）**。在预训练中，因为我们的目标是让模型拥有广泛的知识基础，所以使用覆盖广泛的数据，例如互联网文本。到了 post-training RL，目标变得更窄：我们希望模型在某个具体任务上获得很高准确率，例如解数学题。

这个更窄的目标与预训练至少有两点不同：

1. 我们拥有的数据不再那么多；
2. 训练目标从“coverage（覆盖广泛知识）”变成“precision（生成准确回答）”。

因此，我们需要一种新的技术：强化学习。

在 RL 中，我们会得到一个数据集和一个评分函数，用来判断某个回答是否正确解决了问题。例如，在编程任务中，我们可能得到题目“写一个 Python 函数反转列表”，评分函数由一组测试用例组成，例如：

```
assert f([0, 1, 2]) == [2, 1, 0]
```

在数学任务中，我们可能得到一道文字推理题，例如：“四年前 Tom 的年龄是 John 的一半，而 John 现在 20 岁，Tom 现在几岁？”，评分函数会解析最终答案并判断是否正确，例如答案为 12。

与预训练显著不同的是：RL 中并没有一个现成的“正确回答数据集”让我们模仿。在编程例子里，我们没有给定一份正确 Python 程序；在数学推理例子里，也没有给定一条正确的完整推理链。相反，我们会直接把**模型准确率**作为目标函数，对其做梯度更新。

从高层看，RL 的过程就是：从模型中采样回答，用评分函数给回答打分，然后提高正确回答的权重。

本作业会从数学和实验两个角度学习 RL。RL 实际上相当困难：速度慢、不稳定，而且不同随机种子之间方差很大；一些看似很小的实现细节也可能产生巨大影响。因此，本作业会介绍 LLM RL，并探索其中的一些主要挑战。

2.2 模型与数据集

本作业使用 `OLMo-2-0425-1B`。这是一个 base model，先在 OLMo-mix-1124 上预训练，再在 Dolmino-mix-1124 上进行 mid-training，总训练 token 数为 **4 万亿**。

OLMo-mix-1124 主要由 DCLM-Baseline [J. Li et al., 2024] 构成，这是课程中介绍过的数据集。Dolmino-mix-1124 则是一个更聚焦的数据混合，大约由 50% DCLM 与 50% 指令遵循、数学、代码、STEM 论文及 Wikipedia 数据组成。关于 OLMo-2-0425-1B 的其他细节可参考 OLMo 2 技术报告 [T. OLMo et al., 2024]。到课程目前这个阶段，你应当已经具备理解其中设计选择所需的背景知识。

我们的下游任务使用 GSM8K 数据集 [K. Cobbe et al., 2021]。它位于作业仓库中的：

- `data/gsm8k/train.jsonl`

- `data/gsm8k/test.jsonl`

GSM8K 是一个相对简单的小学数学文字推理题集合。例如：

```
{
  "question": "Natalia sold clips to 48 of her friends in April, and then she sold half as many clips in May. How many clips did Natalia sell altogether in April and May?",
  "answer": "Natalia sold  $48/2 = <<48/2=24>>24$  clips in May.\nNatalia sold  $48+24 = <<48+24=72>>72$  clips altogether in April and May.\n#### 72"
}
```

在 RL 训练期间，模型将学会为这类题生成推理链，从而提升解数学问题的能力。

关于模型和数据集选择的说明。受资源限制，我们只选择一个小规模 model-dataset 组合作为 RL 实验平台。具体来说，我们使用一个小模型，但它在极大量 token 上训练过（token 数约为参数量的 4000 倍），因此能力相对较强，使我们能够在较小规模下、在真实数据集上观察合理的 RL 效果。

本课程中我们自己训练的模型遗憾地还太弱，无法解决这些数学题。如果完成作业后你感兴趣，也可以对自己训练的模型在更简单任务上运行 RL。需要注意的是，RL 的训练动态高度依赖具体模型和数据集，因此本作业观察到的结果不一定能迁移到其他 model-dataset 组合。

2.3 记号

本作业包含较多数学内容。下面列出后文常用记号。语言建模和强化学习经常用不同术语表示同一个对象，因此表中同时给出两套术语。

记号	LM 术语	RL 术语	含义
ρ	prompt distribution / dataset	initial state distribution	prompts/problems 的分布
x	prompt; question	initial state	从 ρ 中采样得到的 prompt/problem
y	response; completion; generation; sample	rollout; trajectory; sampled action sequence	对 prompt x 采样得到的回答
y_t	token	action	y 中第 t 个生成 token
$y_{<t}$	prefix	-	位置 t 之前的所有生成 token，即 $y_1, \dots, y_{\{t-1\}}$
π_θ	model	policy	参数为 θ 的模型；给定 prompt x 时为 response y 分配概率 $\pi_\theta(y x)$
$\pi_\theta(y_t x, y_{<t})$	next-token distribution	policy at timestep t	给定 prompt 和此前生成 token 时，token y_t 的条件概率
$r(y x)$	-	reward	表示采样回答是否正确的标量分数；本作业中为 0 或 1
B	number of prompts per batch	-	每个 inference batch 中的 prompt 数量
G	generations per prompt	group size	每个 prompt 采样的 response 数量
$\text{len}(y)$ 或 L	response length	horizon	一个 response 中生成 token 的数量
$A^{(i,j)}$	-	advantage	在 baseline 与 normalization 后，赋给 prompt i 的 response j 的权重

3 Prompting

把预训练 base model 应用于下游任务时，第一步是给它 prompt。Base model 在预训练阶段学到了广泛的行为，而 prompting 是一种轻量级方法，可以把模型行为引导到“解决任务”的模式。后面还会看到，prompt 的选择也会影响 RL 的训练动态与 exploration。

最基本的 prompting 方法是：直接提供问题，然后从模型的 next-token distribution 中进行采样生成回答。我们把这种策略称为 `question_only`。

我们会把它与 `r1_zero` prompt 比较。`r1_zero` 不仅给出问题，还明确指示模型进行 chain-of-thought 推理 [DeepSeek-AI et al., 2025]。

3.1 使用 vLLM 进行推理

要从模型生成回答，我们需要一个 inference engine。自己实现完整推理引擎超出本作业范围，因此使用 vLLM [W. Kwon et al., 2023]。vLLM 实现了多种优化，包括高速 CUDA kernel、用于高效 attention KV cache 的 PagedAttention 等。

启动 vLLM server 并生成文本的代码已经提供在 `cs336_alignment/vllm_utils.py` 中，接口如下：

```
@dataclass
class VLLMCompletion:
    text: str
    token_ids: list[int]
    finish_reason: str | None
```

```
@dataclass
class VLLMServer:
    model_id: str
    gpu: int = 0
    seed: int = 0
    gpu_memory_utilization: float = 0.9

    def start(self) -> None: ...

    def generate_completions(
        self,
        prompts: list[str],
        sampling_params: dict,
        batch_size: int | None = None,
    ) -> list[VLLMCompletion]: ...
```

3.2 零样本、少样本与思维链提示

除非另有说明，在 GSM8K 实验中我们使用 DeepSeek R1-Zero 模型 [DeepSeek-AI et al., 2025] 的以下 prompt，并称之为 `r1_zero` prompt：

```
User 与 Assistant 进行一段对话。User 提出一个问题，Assistant 解决该问题。
Assistant 首先在内部思考推理过程，然后向 User 给出答案。
推理过程放在 <think> </think> 标签中，答案放在 <answer> </answer> 标签中。
也就是说：<think> reasoning process here </think> <answer> answer here </answer>。

User: {question}
Assistant: <think>
```

该 prompt 位于 `cs336_alignment/prompts/r1_zero.prompt`。

其中，`question` 表示我们插入的问题。例如：“Natalia 四月份向 48 个朋友卖了回形针，五月份卖了四月份的一半，总共卖了多少？”

我们期望模型扮演 Assistant：因为 prompt 中已经包含开头的 `<think>`，模型应直接开始生成思考过程，然后用 `</think>` 关闭思考部分，再在 `<answer>...</answer>` 中生成最终符号答案，例如：

```
<answer> 4x + 10 </answer>
```

让模型使用 `<answer>...</answer>` 的好处是：我们可以更容易解析输出并与 ground-truth answer 比较，同时看到 `</answer>` 后就可以停止生成。

另一种方法叫少样本提示（few-shot prompting）：在真正问题之前加入若干组 question-answer 示例。`r1_zero` 的 few-shot 版本大致如下：

User 与 Assistant 进行一段对话。User 提出一个问题，Assistant 解决该问题。
Assistant 首先进行推理，然后给出答案。
推理过程放在 `<think>` `</think>` 中，答案放在 `<answer>` `</answer>` 中。

```
User: {question-1}
Assistant: <think> {reasoning-1} </think> <answer> {answer-1} </answer>
User: {question-2}
Assistant: <think> {reasoning-2} </think> <answer> {answer-2} </answer>
User: {question-3}
Assistant: <think> {reasoning-3} </think> <answer> {answer-3} </answer>
User: {question}
Assistant: <think>
```

Few-shot prompting 通过给模型几个任务示例来提高表现。公开 base model 的 benchmark 指标经常是在 few-shot 设置下报告的。例如，OLMo-2-0425-1B model card 上的 GSM8K 结果使用的是 8-shot prompting。

作业已提供 `r1_zero` 的 3-shot 版本：

```
cs336_alignment/prompts/r1_zero_three_shot_gsm8k.prompt
```

其中示例来自 OLMES 仓库。

最后，我们还包括一个 baseline: `question_only`，位于：

```
cs336_alignment/prompts/question_only.prompt
```

```
{question} Please put your final answer within \boxed{{}}.
```

虽然叫 `question_only`，但我们仍要求最终答案放在 `\boxed{{}}` 中，这样 grader 更容易从 response 中解析答案。

3.3 评分函数

模型生成回答后，需要判断回答是否正确。数学题有 ground-truth answer，例如 `0.5`，但模型可能用多种方式表达同一个正确答案，例如：

```
<answer> 1/2 </answer>
```

或者：

```
The answer is 0.5.
```

因此，要正确给模型输出评分，我们需要一个 answer parsing 函数：输入模型输出和已知 ground truth，返回一个布尔值表示是否正确。

本作业使用近期 reasoning RL 工作 [Z. Liu et al., 2025] 中的一种快速且较准确的答案解析器。

对于 `r1_zero` prompt，对应 reward function 位于：

```
cs336_alignment.drgrpо_grader.r1_zero_reward_fn
```

`question_only` prompt 不要求模型使用 `<think>` 和 `<answer>` 标签，所以应使用同一 grader 文件中的：

```
cs336_alignment.drgrpо_grader.question_only_reward_fn
```

`question_only` parser 会寻找 `\boxed{{}}` 中的最终答案。

这些 reward function 会返回：

- total reward
- format reward: 输出是否符合预期格式
- answer reward: 解析出的答案是否正确

有些工作会给“格式正确但答案错误”的回答部分分数，也就是非零 total reward。本作业**不使用部分奖励**，所以 total reward 就等于 answer reward；format reward 只用于 logging。

注意，这些 reward function 期望 `ground_truth` 参数中只包含最终答案。GSM8K 的原始 ground-truth response 格式为：

```
{rationale} #### {answer}
```

因此，应对字符串按 `####` 切分并去除空白，从而得到最终答案。

3.4 实验

现在可以评估 base model 的 prompting 性能。

生成超参数。生成 response 时使用：

- temperature = `1.0`
- top-p = `1.0`
- 最大生成长度 = `512`

`r1_zero` prompt 要求模型以 `</answer>` 结束答案，所以使用这些 prompt 时，可以让 vLLM 在生成这个字符串后停止：

```
# Based on Dr. GRPO: stop when the model completes its answer
# https://github.com/sail-sg/understand-r1-zero/blob/
# c18804602b85da9e88b4aeeb6c43e2f08c594fbc/train_zero_math.py#L167
sampling_params['stop'] = ["</answer>"]
sampling_params['include_stop_str_in_output'] = True
```

只对 `r1_zero` 和 `r1_zero_three_shot` 使用这个 stop string；不要对 `question_only` 使用。

题目 (prompting baselines)：在 GSM8K 上运行 OLMo-2-0425-1B (5 分)

(a) 编写脚本，使用以下三种 prompt 评估 OLMo-2-0425-1B 在 GSM8K 上的性能：

- zero-shot `question_only`
- zero-shot `r1_zero`
- few-shot `r1_zero_three_shot`

然后运行脚本并观察输出。对于每种 prompt，模型生成结果分别有多少落入以下三类：

1. format reward = 1 且 correctness reward = 1，即格式和答案都正确；
2. format reward = 1，但 correctness reward = 0；
3. format reward = 0 且 correctness reward = 0。

至少观察类别 2 中 10 个样例：其中有多少模型输出其实是正确的，只是 parser 没有正确解析？类别 3 又如何？

交付内容：几句评论、评测指标，以及若干 prompt 和 response 示例。

(b) 观察模型输出，描述不同 prompt 下的模型行为。例如，如果我们希望模型回答问题，仅仅给出问题本身是否足够？模型是否会表现出其他行为，而不只是回答问题？zero-shot `r1_zero` 和 few-shot `r1_zero_three_shot` 是如何改变模型行为的？

交付内容：几句评论，并给出支持性示例。

4 Group Relative Policy Optimization (GRPO, 组相对策略优化)

在只使用 prompting 测量过模型性能之后，下一步是通过训练进一步提高模型表现。具体来说，我们希望优化模型的准确率，也就是最大化期望奖励：

$$J_{\theta} = \mathbb{E}_{x \sim \rho} \mathbb{E}_{y \sim \pi_{\theta}(y|x)} [r(y|x)]. \quad (1)$$

其中， ρ 是 prompt / 问题 x 的任务分布， π_{θ} 是模型， y 是针对 x 采样得到的 response / solution， $r(y|x)$ 表示 y 是否正确解决了 x 。后面也会把 r 称为 **reward function (奖励函数)**。

这个目标与此前学习的预训练目标有很大不同。预训练通常最小化交叉熵：

$$\mathbb{E}_{x \sim \mathcal{D}} [-\log \pi_{\theta}(x)], \quad (1)$$

样本来自固定数据集 $\mathcal{D}$ 。而这里的准确率目标中，response 是由模型自己采样出来的。因此，我们需要新的优化工具，也就是 reinforcement learning (RL, 强化学习)。从宏观上看，RL 会反复进行：

1. 从模型采样 response；
2. 用评分函数给 response 打分；
3. 强化那些正确的 response。

RL 往往速度慢、训练困难，因此本作业的大量内容都会围绕如何让训练更快、更稳定展开。

另一个重要点是：RL 依赖从模型中采样，因此训练动态会受到 prompt 的影响。语言模型研究中的一个重要现象是：对较强的 base model 使用 chain-of-thought reasoning prompt，再进行 RL，可能显著提高推理能力 [OpenAI et al., 2024; DeepSeek-AI et al., 2025]。所以本作业除了研究 RL 的核心算法，也会研究 RL 与 CoT prompting 的相互作用。

4.1 推导 on-policy GRPO

首先从 RL 的数学基础开始，一步一步推导 Group Relative Policy Optimization (GRPO)。GRPO 是训练语言模型时常见的一种 RL 算法 [Z. Shao et al., 2024]。

本节的讲解主要参考了两个资源：OpenAI 的 *Spinning Up in Deep RL* [J. Achiam, 2018] 和 Nathan Lambert 的 *Reinforcement Learning from Human Feedback (RLHF) Book* [N. Lambert, 2024]。

4.1.1 把语言模型看作 policy

传统强化学习中，可以把 policy 理解为一个函数：给定状态 s_t ，输出动作 a_t 。执行动作以后，会得到即时奖励

$$r_t = r(s_t, a_t), \quad (2)$$

并根据某个 next-state distribution 转移到后续状态：

$$s_{t+1} \sim \text{next_state}(s_t, a_t). \quad (3)$$

强化学习的目标就是优化 policy，使累计奖励尽可能高。

对于参数为 θ 的 causal language model π_{θ} ，给定当前文本前缀

$$y_{<t} = (y_1, \dots, y_{t-1}), \quad (4)$$

模型会定义下一个 token y_t 的概率分布。因此，在 RL 语言中，可以对应为：

- 当前文本 prefix $y_{<t}$ = state s_t
- 下一个 token y_t = action a_t

于是语言模型就是一个 categorical stochastic policy:

$$a_t \sim \pi_\theta(\cdot | s_t), \quad \pi_\theta(a_t | s_t) = [\text{softmax}(f_\theta(s_t))]_{a_t}. \quad (2)$$

使用 policy gradient 优化 policy 时, 我们至少需要两个基本操作:

1. 从 **policy** 采样: 从上述 categorical distribution 中采样动作 a_t ;
2. 计算某个动作的 **log-likelihood**: 计算 $\log \pi_\theta(a_t | s_t)$ 。

在 LLM RL 中, s_t 通常就是当前已经生成的部分 completion / solution, a_t 就是下一个 token。当模型生成 end-of-text token, 例如 `<|end_of_text|>`, 或者在本作业的 `r1_zero` prompt 中生成 `</answer>` 时, 一个 episode 就结束。

4.1.2 Trajectory (轨迹)

在强化学习中, 我们首先从初始状态分布中采样:

$$s_0 \sim \rho. \quad (5)$$

然后不断重复:

1. 从 policy π_θ 中采样动作 a_t ;
2. 根据 next-state distribution 得到下一状态 s_{t+1} ;
3. 继续采样下一个动作。

这串状态和动作构成一个有限长度的 trajectory:

$$\tau = (s_0, a_0, s_1, a_1, \dots, s_T, a_T). \quad (3)$$

其中 T 是 trajectory 的长度; 也就是说, a_T 要么是 end-of-text token, 要么是因为达到最大 token generation budget 而终止。

在本作业里:

- $s_0 \sim \rho$ 就是包含数学题的 prompt x , 它可能已经应用 CoT 或 few-shot prompting;
- 从这个 prefix 开始, 每次采样一个 token;
- 下一状态就是旧 prefix 加上刚刚生成的 token:

$$s_{t+1} = (s_t, a_t). \quad (6)$$

Trajectory 也经常称为 **episode** 或 **rollout**。本作业中这些术语会交替使用。

4.1.3 Reward 与 return

RL 中, 标量 reward

$$r_t = r(s_t, a_t) \quad (7)$$

表示在状态 s_t 下执行动作 a_t 的即时质量。

对于具有可验证答案的任务, 例如这里的数学应用题, 在中间推理步骤中通常不给奖励; 直到输出最终答案时, 才在 terminal action 上获得可验证的 reward:

$$r_T = r(s_T, a_T) := \begin{cases} 1, & \text{如果 trajectory } \tau \text{ 根据 reward function 与 ground truth 匹配,} \\ 0, & \text{否则.} \end{cases} \quad (4)$$

因此, 在本作业中:

- 最终答案正确: $r_T = 1$;
- 最终答案错误: $r_T = 0$ 。

Return $R(\tau)$ 用来汇总整条 trajectory 上的 reward。在本作业中，中间 reward 都是 0，因此

$$R(\tau) = r_T. \quad (8)$$

Agent 的目标是最大化期望 return：

$$J_\theta = \mathbb{E}_{\tau \sim \pi_\theta}[r(\tau)]. \quad (5)$$

这里的 $\tau \sim \pi_\theta$ 表示：先采样 $s_0 \sim \rho$ ，然后不断从 policy 采样动作并转移状态。

对语言模型而言，就是先从数据集中采样数学题

$$x \sim \rho, \quad (9)$$

再从模型生成 response：

$$y \sim \pi_\theta(y | x). \quad (10)$$

最终得到优化问题：

$$\theta^* = \arg \max_{\theta} J_\theta. \quad (6)$$

4.1.4 Policy gradient

现在已经具备推导 policy gradient 所需的符号和定义。接下来统一使用语言模型符号 (x, y) ，而不再主要使用 RL 的 state/action 符号 (s_t, a_t) 。

我们的目标仍然是最大化模型的期望 reward：

$$J_\theta = \mathbb{E}_{x \sim \rho} \mathbb{E}_{y \sim \pi_\theta(y|x)}[r(y | x)]. \quad (7)$$

一种自然做法是对该目标做 gradient ascent：

$$\theta_{k+1} = \theta_k + \alpha \nabla_{\theta} J_{\theta_k}. \quad (8)$$

问题在于，我们需要能从有限样本估计 $\nabla_{\theta} J_{\theta}$ 。把这个梯度重新写成关于模型样本的期望，就会得到著名的 **REINFORCE policy gradient**：

$$\nabla_{\theta} \mathbb{E}_{x \sim \rho} \mathbb{E}_{y \sim \pi_\theta(y|x)}[r(y | x)] = \mathbb{E}_{x \sim \rho} \mathbb{E}_{y \sim \pi_\theta(y|x)}[r(y | x) \nabla_{\theta} \log \pi_\theta(y | x)]. \quad (9)$$

这个式子直接给出了一个很直观的算法：

1. 从数据集中采样问题 x ；
2. 从模型中采样 response y ；
3. 计算 $r(y | x) \nabla_{\theta} \log \pi_\theta(y | x)$ ；
4. 在一批样本上取平均；
5. 用这个估计进行梯度更新。

其中

$$r(y | x) \nabla_{\theta} \log \pi_\theta(y | x) \quad (11)$$

可以理解为：**reward 越高的 response，其 log probability 应该被提高得越多。**

反复执行上述步骤，就得到一个基础的强化学习训练循环。

REINFORCE policy gradient 的推导依赖 **log-derivative trick**。因为

$$\nabla_{\theta} \log f(\theta) = \frac{\nabla_{\theta} f(\theta)}{f(\theta)}, \quad (12)$$

所以有

$$\nabla_{\theta} \pi_{\theta}(y | x) = \pi_{\theta}(y | x) \nabla_{\theta} \log \pi_{\theta}(y | x). \quad (10)$$

直接应用这个恒等式：

$$\begin{aligned} \nabla_{\theta} \mathbb{E}_{y \sim \pi_{\theta}(y|x)} [r(y | x)] &= \sum_y \nabla_{\theta} \pi_{\theta}(y | x) r(y | x) \\ &= \sum_y \pi_{\theta}(y | x) \nabla_{\theta} \log \pi_{\theta}(y | x) r(y | x) \\ &= \mathbb{E}_{y \sim \pi_{\theta}(y|x)} [r(y | x) \nabla_{\theta} \log \pi_{\theta}(y | x)]. \end{aligned} \quad (11--13)$$

这里省略了对 prompt 的期望 $\mathbb{E}_{x \sim \rho}$ ，因为 ∇_{θ} 与这个期望可以交换顺序。

假设一批 prompt 为

$$x^{(1)}, \dots, x^{(B)} \stackrel{iid}{\sim} \rho, \quad (13)$$

每个 prompt 采样 G 个 response：

$$y^{(i,j)} \stackrel{iid}{\sim} \pi_{\theta}(y | x^{(i)}), \quad (14)$$

那么 policy gradient 的样本估计为：

$$\hat{g} \leftarrow \frac{1}{BG} \sum_{i=1}^B \sum_{j=1}^G r(y^{(i,j)} | x^{(i)}) \nabla_{\theta} \log \pi_{\theta}(y^{(i,j)} | x^{(i)}). \quad (14)$$

这个 estimator 会提高高 reward response 的概率。接下来推导 GRPO 时，将从这个最基础的 estimator 出发，一项一项加入修改。

4.1.5 Baseline

虽然基础 REINFORCE estimator 的期望是正确的 $\nabla_{\theta} J_{\theta}$ ，但它的方差很大，这会导致 RL 训练不稳定。降低方差的一种常见工具是 baseline。

最简单的 baseline 是从 reward 中减去一个常数 b ：

$$\mathbb{E}_{x \sim \rho} \mathbb{E}_{y \sim \pi_{\theta}(y|x)} [(r(y | x) - b) \nabla_{\theta} \log \pi_{\theta}(y | x)]. \quad (15)$$

例如，当 reward 是二元的（正确为 1、错误为 0），并取 $b = 0.5$ 时：

- 正确 response 的权重变成 +0.5，因此提高其概率；
- 错误 response 的权重变成 -0.5，因此降低其概率。

而原始 estimator 对正确 response 的权重是 1，对错误 response 的权重是 0，也就是只强化正确答案，而不会主动压低错误答案。

只要 baseline b 不依赖于动作 / response y ，减去 baseline 不会改变 estimator 的期望。它可以是常数，也可以依赖于 x 或 policy，但不能依赖当前采样的 response：

$$\mathbb{E} [(r - b) \nabla_{\theta} \log \pi_{\theta}] = \mathbb{E} [r \nabla_{\theta} \log \pi_{\theta}]. \quad (16)$$

这是因为：

$$\mathbb{E}_{y \sim \pi_{\theta}(y|x)} [\nabla_{\theta} \log \pi_{\theta}(y | x)] = \nabla_{\theta} \mathbb{E}_{y \sim \pi_{\theta}(y|x)} [1] = 0. \quad (17)$$

Baseline 虽然保持期望不变，但可能降低也可能增加方差。如果把 policy-gradient estimator 写成样本平均

$$\frac{1}{n} \sum_{i=1}^n Z_i, \quad (15)$$

其中

$$Z_i = (r(y | x) - b) \nabla_{\theta} \log \pi_{\theta}(y | x), \quad (16)$$

则 estimator 的方差为

$$\frac{\mathbb{E}[Z_i^2] - \mathbb{E}[Z_i]^2}{n}. \quad (17)$$

下面的题目会分析 baseline 在什么情况下能真正降低方差。

题目 (baseline_calcs) : 计算 policy-gradient estimator 的方差 (5 分)

设 π_{θ} 是定义在二元动作空间

$$\mathcal{A} = \{0, 1\} \quad (18)$$

上的 policy, 并且

$$\pi_{\theta}(A = 1) = p = \sigma(\theta), \quad \sigma(\theta) = \frac{1}{1 + e^{-\theta}}. \quad (19)$$

二元 reward function 定义为: 动作 $A = 1$ 得到 reward 1, 否则为 0:

$$r(A) = \mathbf{1}\{A = 1\}. \quad (20)$$

(a) 给定 n 个独立同分布样本

$$A_i \stackrel{iid}{\sim} \pi_{\theta}, \quad (21)$$

policy-gradient estimator 为

$$\frac{1}{n} \sum_{i=1}^n r(A_i) \nabla_{\theta} \log \pi_{\theta}(A_i). \quad (18)$$

它的方差是多少?

交付内容: 用 n 和 p 表示的公式, 并附推导。

(b) 如果使用 baseline-adjusted estimator:

$$\frac{1}{n} \sum_{i=1}^n (r(A_i) - b) \nabla_{\theta} \log \pi_{\theta}(A_i), \quad (19)$$

它的方差是多少?

交付内容: 用 n, b, p 表示的公式, 并附推导和一些讨论。

(c) 如果代入“population mean” baseline $b = p$, 最终方差是多少? 与未调整的 policy-gradient estimator 相比, 它的方差是始终更低、始终更高, 还是会根据 p 的不同而有时更高、有时更低?

GRPO 中的 baseline

GRPO 首先采用 **group mean baseline (组均值基线)** :

$$\hat{g} \leftarrow \frac{1}{BG} \sum_{i=1}^B \sum_{j=1}^G \left(r(y^{(i,j)} | x^{(i)}) - \mu_i \right) \nabla_{\theta} \log \pi_{\theta}(y^{(i,j)} | x^{(i)}), \quad (20)$$

其中

$$\mu_i = \frac{1}{G} \sum_{j=1}^G r(y^{(i,j)} | x^{(i)}) \quad (22)$$

就是模型在 prompt $x^{(i)}$ 上这一组 response 的平均 reward。

注意， μ_i 的确依赖 sampled responses y ，所以严格来说它不是前面所说的“与 response 无关”的普通 baseline。但仍然可以证明，它只会让期望多一个缩放因子：

$$\mathbb{E} \left[\frac{1}{G} \sum_{j=1}^G (r(y^{(j)} | x) - \mu_i) \nabla_{\theta} \log \pi_{\theta}(y^{(j)} | x) \right] = \frac{G-1}{G} \mathbb{E} [r(y | x) \nabla_{\theta} \log \pi_{\theta}(y | x)]. \quad (21--24)$$

因此它仍然保留了正确的 policy-gradient 方向，只差一个 $(G-1)/G$ 的常数缩放。

减去 group mean 后得到的 reward 往往称为 **advantage (优势)**：它不再表示 response 的绝对 reward，而表示这个 rollout 相对于同一 prompt 下平均 rollout 的“相对好坏”。

4.1.6 Advantage normalization

GRPO 在减去 group mean 之后，还会进一步除以组内 reward 的标准差：

$$\hat{g} \leftarrow \frac{1}{BG} \sum_{i=1}^B \sum_{j=1}^G \frac{r(y^{(i,j)} | x^{(i)}) - \mu_i}{\text{std}_i} \nabla_{\theta} \log \pi_{\theta}(y^{(i,j)} | x^{(i)}). \quad (25)$$

其中概念上可写成：

$$\text{std}_i = \sqrt{\frac{1}{G} \sum_{j=1}^G (r(y^{(i,j)} | x^{(i)}) - \mu_i)^2}. \quad (23)$$

实现注意： 默认的 `torch.std` 会使用略有不同的 sample standard deviation 形式进行 bias correction。本作业要求实现时直接使用默认 `torch.std`。

与减去 baseline 不同，除以标准差不再保持原 **estimator** 的期望，因此此时严格来说已经不再是在原始期望 reward 目标 J_{θ} 上做精确的 gradient ascent。

一种理解方式是：这是一个稳定性技巧。若粗略假设各 gradient vector 是 iid Gaussian，那么除以组内标准差，会让不同 group 的 gradient update 在 norm 上大致处于相近尺度。

4.1.7 Sequence normalization

GRPO 还有一个似乎继承自早期 LLM-PPO 实现的细节：**sequence normalization (序列归一化)**。

首先，把 response 的 log probability 展开成逐 token 求和，则前面的 estimator 可以写为：

$$\hat{g} \leftarrow \frac{1}{BG} \sum_{i=1}^B \sum_{j=1}^G \frac{r(y^{(i,j)} | x^{(i)}) - \mu_i}{\text{std}_i} \nabla_{\theta} \sum_{t=1}^{\text{len}(y^{(i,j)})} \log \pi_{\theta}(y_t^{(i,j)} | x^{(i)}, y_{<t}^{(i,j)}). \quad (26)$$

GRPO 额外加入一个 response-length normalization：

$$\hat{g} \leftarrow \frac{1}{BG} \sum_{i=1}^B \sum_{j=1}^G \frac{r(y^{(i,j)} | x^{(i)}) - \mu_i}{\text{std}_i} \nabla_{\theta} \left(\frac{1}{\text{len}(y^{(i,j)})} \sum_{t=1}^{\text{len}(y^{(i,j)})} \log \pi_{\theta}(y_t^{(i,j)} | x^{(i)}, y_{<t}^{(i,j)}) \right). \quad (27)$$

这项修改会进一步改变 estimator 的期望：它不再让所有 sequence 中的 token 拥有相同权重，而是**相对于短 sequence，降低长 sequence 中每个 token 的权重**。

下一节会进一步讨论这个设计，并通过 ablation 检查它是否是好选择。不过目前先实现原论文中的标准 GRPO，所以仍然使用 sequence normalization。

4.1.8 把所有部分组合起来

GRPO 还有一个尚未讨论的组成部分：**clipped importance reweighting**（裁剪的重要性重加权）。它会在后面的 off-policy RL 部分详细介绍。

简单来说：

- **on-policy RL**：每个 inference batch 只进行一次 gradient update；
- **off-policy RL**：同一个 inference batch 上进行多次 gradient update，以提高训练速度。

从第二次更新开始，inference batch 中的样本就已经来自“旧模型”，因此变成 stale sample。Importance reweighting 可以调整这些 stale sample 的权重，使它们更适合当前 policy。

本节暂时只做 on-policy RL，所以先不需要 importance reweighting。

现在已经具备实现 on-policy GRPO 的全部组件。完整算法如下。

算法 1：On-policy Group Relative Policy Optimization (GRPO)

输入：初始 policy model π_{θ_0} 、reward function r 、task distribution / dataset ρ 、learning rate α 。

输出：policy model π_{θ} 。

1. 初始化 $\pi_{\theta} \leftarrow \pi_{\theta_0}$ 。
2. 对 `step = 1, ..., n_grpo_steps` :
 1. 从 ρ 采样一批 B 个问题：

$$x^{(1)}, \dots, x^{(B)} \stackrel{iid}{\sim} \rho. \quad (24)$$

2. 对每个问题采样 G 个输出：

$$y^{(i,1)}, \dots, y^{(i,G)} \stackrel{iid}{\sim} \pi_{\theta}(y \mid x^{(i)}). \quad (25)$$

3. 计算 on-policy GRPO policy-gradient estimator：

$$\hat{g} \leftarrow \frac{1}{BG} \sum_{i=1}^B \sum_{j=1}^G \frac{1}{\text{len}(y^{(i,j)})} \sum_{t=1}^{\text{len}(y^{(i,j)})} \frac{r(y^{(i,j)} \mid x^{(i)}) - \mu_i}{\text{std}_i} \nabla_{\theta} \log \pi_{\theta}(y_t^{(i,j)} \mid x^{(i)}, y_{<t}^{(i,j)}). \quad (28)$$

其中 group mean 为

$$\mu_i = \frac{1}{G} \sum_{j=1}^G r(y^{(i,j)} \mid x^{(i)}), \quad (29)$$

group standard deviation 为

$$\text{std}_i = \sqrt{\frac{1}{G} \sum_{j=1}^G (r(y^{(i,j)} \mid x^{(i)}) - \mu_i)^2}. \quad (30)$$

4. 使用任意 optimizer 更新 θ 。如果写成 stochastic gradient ascent：

$$\theta \leftarrow \theta + \alpha \hat{g}. \quad (31)$$

4.2 实现 on-policy GRPO

4.2.1 使用 Hugging Face 模型

此前的作业使用我们在 `cs336_basics` 中自行实现的 language model。本作业则直接使用 Hugging Face `transformers` 库加载预训练 base model。如果愿意，你也可以自己实现 transformer 和预训练权重加载器，但需要保证模型结构与 OLMo-2-0425-1B 完全匹配。

为了使用 bfloat16 并通过 FlashAttention-2 节省显存，可以使用 `cs336_alignment/checkpoint.py` 中提供的 starter code：

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer

def get_model_and_tokenizer(model_id_or_dir: str, device: str):
    model = AutoModelForCausalLM.from_pretrained(
        model_id_or_dir,
        device_map=device,
        torch_dtype=torch.bfloat16,
        attn_implementation="eager" if device == 'cpu' else "flash_attention_2",
    )
    tokenizer = AutoTokenizer.from_pretrained(model_id_or_dir)
    return model, tokenizer
```

`model_id_or_dir` 可以是模型名称，例如 `allenai/OLMo-2-0425-1B`，也可以是一个本地目录路径。这个目录通常来自 `save_pretrained`：

```
# Save the model weights
model.save_pretrained(save_directory=output_dir)
tokenizer.save_pretrained(save_directory=output_dir)
```

接下来先实现一个 helper function，用预训练 tokenizer 对 prompt 和 response 进行 tokenize。

要求是：

1. prompt 和 response 分别 **tokenize**；
2. 不添加 special token；
3. 直接拼接为 `prompt_ids + response_ids`；
4. prompt 与 response 之间不插入 EOS、BOS 或 separator token；
5. 构造 `response_mask`。

`response_mask` 是一个与 shifted labels 对齐的 boolean mask：

- 如果某个 label token 来自 response，则为 `True`；
- 如果来自 prompt 或 padding，则为 `False`。

训练时会使用这个 mask，确保 loss 只计算 **response token**。

题目 (`tokenize_prompt_and_output`)：Prompt 与 output tokenization (1 分)

实现 `tokenize_prompt_and_output`：分别 tokenize prompt 和 output，将它们直接拼接，并构造 `response_mask`。

推荐接口：

```
def tokenize_prompt_and_output(
    prompt_strs: list[str],
    output_strs: list[str],
    tokenizer: PreTrainedTokenizer,
) -> dict[str, torch.Tensor]:
```

参数:

- `prompt_strs: list[str]`: prompt 字符串列表;
- `output_strs: list[str]`: output 字符串列表;
- `tokenizer: PreTrainedTokenizer`: 用于 tokenization 的 tokenizer。

设 `prompt_and_output_lens` 为每个样本拼接后 token 序列长度的列表。返回 dictionary 应包含:

- `input_ids`: shape 为 `(batch_size, max(prompt_and_output_lens) - 1)`; 即拼接后的 token 序列去掉最后一个 token。
- `labels`: shape 相同; 即 shifted input IDs, 也就是去掉第一个 token。
- `response_mask`: shape 相同; 与 `labels` 对齐, response 对应的 label token 为 1, prompt 和 padding 位置为 0。

测试方法: 在 `tests/adapters.py` 中实现 `adapters.run_tokenize_prompt_and_output`, 然后运行:

```
uv run pytest -k test_tokenize_prompt_and_output
```

模型前向可以写成:

```
input_ids = train_batch["input_ids"].to(device)
labels = train_batch["labels"].to(device)
logits = model(input_ids).logits
```

下一步实现一个函数, 计算 response 中每个 token 在模型下的 conditional log-probability。这是计算 policy gradient 的核心 primitive。在 RL 中也常常记录 per-token entropy, 因此函数还需要支持 `return_token_entropy`。

题目 (`get_response_log_probs`): Response 的 log-probability 与 entropy (1 分)

实现 `get_response_log_probs`, 得到 causal language model 对每个 token 的条件 log-probability, 并可选返回 next-token distribution 的 entropy。

推荐接口:

```
def get_response_log_probs(
    model: PreTrainedModel,
    input_ids: torch.Tensor,
    labels: torch.Tensor,
    return_token_entropy: bool = False,
) -> dict[str, torch.Tensor]:
```

参数:

- `model`: 用于打分的 Hugging Face `PreTrainedModel`; 模型应已经放到正确 device。如果不需要计算 gradient, 应处于 inference mode。
- `input_ids`: shape `(batch_size, sequence_length)`, 为前面 tokenization 方法得到的 prompt + response

token。

- `labels`: shape `(batch_size, sequence_length)`, 为前面 tokenization 方法得到的 labels。
- `return_token_entropy`: 如果为 `True`, 同时返回每个 token 的 entropy。

返回:

- `"log_probs"`: shape `(batch_size, sequence_length)`, 表示

$$\log p_{\theta}(x_t | x_{<t}). \quad (26)$$

- `"token_entropy"`: 可选, shape 相同, 仅在 `return_token_entropy=True` 时出现。

测试:

```
uv run pytest -k test_get_response_log_probs
```

4.2.2 在 RL loop 中使用 vLLM

RL loop 还需要生成 rollouts。本作业使用的具体硬件配置是:

- 一张 GPU 放 Hugging Face policy model 和 optimizer, 用于训练;
- 另一张 GPU 运行 vLLM, 包括模型本身和 KV cache, 用于 inference。

因此, 每次 inference 之前, 还需要把训练 GPU 上最新的 policy weights 同步到 vLLM GPU。

`cs336_alignment/vllm_utils.py` 中已经提供了相应的 weight sync 功能:

```
@dataclass
class VLLMServer:
    gpu: int = 1 # Run training on gpu 0 and inference on gpu 1

    # Create the NCCL weight-transfer group between the training GPU and vLLM.
    def init_weight_sync(self, policy_device: str): ...

    # Copy the current Hugging Face policy weights into the vLLM server and
    # reset vLLM caches that depended on the old weights.
    def sync_policy_weights(self, policy: torch.nn.Module) -> None: ...

    # Generate rollouts from the current vLLM weights.
    def generate_completions(
        self,
        prompts: list[str],
        sampling_params: dict,
        batch_size: int | None = None,
    ) -> list[VLLMCompletion]: ...
```

与 prompting 实验一致, 采样参数使用:

- `temperature = 1.0`
- `top-p = 1.0`
- `最大生成长度 = 512`

因为 prompt 要求答案以 `</answer>` 结尾, 可以让 vLLM 在该字符串出现时停止:

```
sampling_params['stop'] = ["</answer>"]
sampling_params['include_stop_str_in_output'] = True
```

4.2.3 GRPO 的组成部分

接下来分别实现 GRPO loss 的各个组成部分。后面还会实现不同的 advantage normalizer 与 importance reweighting 变体，因此这里的函数接口有意设计成可方便切换 variant。

当前阶段只需要实现标准 GRPO。

题目 (compute_rollout_rewards)：计算 rollout reward (1 分)

实现 `compute_rollout_rewards`，对每个 rollout response 计算原始 reward。

推荐接口：

```
def compute_rollout_rewards(
    reward_fn: Callable[[str, str], dict[str, float]],
    rollout_responses: list[str],
    repeated_ground_truths: list[str],
) -> tuple[torch.Tensor, dict[str, float]]:
```

功能：为一批 rollout responses 计算 reward，同时返回各 reward component 的 metadata。

参数：

- `reward_fn`：输入 response 和 ground truth，输出包含 "reward"、"format_reward"、"answer_reward" 的 dict。
- `rollout_responses`：policy 生成的 rollouts。长度为 `rollout_batch_size = n_prompts_per_rollout_batch * group_size`。
- `repeated_ground_truths`：对应 ground truth。因为同一个问题的 ground truth 会重复 `group_size` 次，所以长度也是 `rollout_batch_size`。

返回：

- `raw_rewards`：shape `(rollout_batch_size,)`，每个 rollout 的未归一化 reward；
- `metadata`：用于 logging 的 reward statistics。至少包括整个 rollout batch 的 mean total reward 和 mean format reward。

测试：

```
uv run pytest -k compute_rollout_rewards
```

下一步实现 GRPO 的核心数学：把 raw reward 在每个 group 内归一化，得到 advantage。

由于上一函数输出的 reward 已经 flatten 成一维，因此需要通过 `group_size` reshape 回各组再归一化。

为了通过当前测试，需要支持：

- `baseline = "mean"`
- `advantage_normalizer = "std"`

为了防止除以 0，在每组标准差上加 `advantage_eps`。

题目 (compute_group_normalized_rewards_grpo)：Group normalization (1 分)

实现 `compute_group_normalized_rewards`，对每组 raw reward 进行归一化并返回 advantage。

推荐接口：

```
def compute_group_normalized_rewards(
    raw_rewards: torch.Tensor,
    group_size: int,
    baseline: Literal["mean", "none"] = "mean",
    advantage_eps: float = 1e-6,
    advantage_normalizer: Literal["std", "none", "mean"] = "std",
):
```

参数：

- `raw_rewards`：shape `(rollout_batch_size,)`；
- `group_size`：每个 question 的 response 数量；
- `baseline`：当前题目只需支持 `"mean"`，即减去组内平均 reward；后面 `"none"` 表示不减 baseline；
- `advantage_eps`：用于避免除零的小常数；
- `advantage_normalizer`：当前只需支持 `"std"`，即除以组内标准差。后面还会支持 `"none"` 与 `"mean"`。

返回：

- `advantages`：shape `(rollout_batch_size,)`；
- `metadata`：你认为有用的统计量，例如 reward 的 mean/std/max/min。

对于当前未支持的选项，可以直接 `raise NotImplementedError`。

测试：

```
uv run pytest -k compute_group_normalized_rewards_grpo
```

接下来实现逐 token 的 **policy-gradient loss**。

Algorithm 1 中写的是 gradient estimator，但实际使用 PyTorch 时，需要构造一个 loss，使得对它求 gradient 可以产生对应的 policy-gradient term。

标准 on-policy GRPO 的 objective 为：

$$J_{\theta}^{\text{GRPO-on-policy}} = \frac{1}{BG} \sum_{i=1}^B \sum_{j=1}^G \frac{1}{\text{len}(y^{(i,j)})} \sum_{t=1}^{\text{len}(y^{(i,j)})} \frac{r(y^{(i,j)} | x^{(i)}) - \mu_i}{\text{std}_i} \log \pi_{\theta}(y_t^{(i,j)} | x^{(i)}, y_{<t}^{(i,j)}). \tag{32}$$

这个表达式并不是传统意义上“训练过程中应不断下降”的 loss，而只是一个求导后会得到我们想要的 policy gradient 的 objective。

由于 PyTorch optimizer 默认做 gradient descent，因此实现时应返回这个 objective 的负值。

`compute_policy_gradient_loss` 负责计算每个 token、每个 sequence 的 loss term；随后 `aggregate_loss_across_microbatch` 再在 token 和 sequence 维度上做 aggregation。

题目（`compute_policy_gradient_loss_on_policy`）：On-policy policy gradient（1 分）

实现 `compute_policy_gradient_loss`，给定 raw rewards 或预先计算好的 advantages，计算逐 token policy-gradient loss。

当前所有 rollout 都是 on-policy，因此只需要支持：

```
importance_reweighting_method = "none"
```

此时可以忽略 old-log-prob 和 clipping 参数。对于其他选项，可以 `raise NotImplementedError`。

推荐接口：

```
def compute_policy_gradient_loss(
    raw_rewards_or_advantages: torch.Tensor,
    policy_log_probs: torch.Tensor,
    importance_reweighting_method: Literal["none", "noclip", "grpo", "gspo"] = "none",
    old_log_probs: torch.Tensor | None = None,
    cliprange: float | None = None,
    response_mask: torch.Tensor | None = None,
) -> tuple[torch.Tensor, dict[str, torch.Tensor]]:
```

参数：

- `raw_rewards_or_advantages`：shape `(batch_size,)` 或 `(batch_size, 1)`；每个 rollout 对应一个标量 reward / advantage；
- `policy_log_probs`：shape `(batch_size, sequence_length)`；每个 token 的 log-probability；
- `importance_reweighting_method`：
 - `"none"`：无 importance reweighting；
 - `"noclip"`：importance reweighting，但不 clipping；
 - `"grpo"`：PPO/GRPO 风格 token-level reweighting + clipping；
 - `"gspo"`：GSPO 风格 sequence-level reweighting + clipping。
- `old_log_probs`：除 `"none"` 外均需要，shape `(batch_size, sequence_length)`；
- `cliprange`：裁剪参数 ϵ ，当 method 为 `"grpo"` 或 `"gspo"` 时需要；
- `response_mask`：可选，shape `(batch_size, sequence_length)`。GSPO 需要只在 response token 上平均 sequence-level log-ratio，因此会用到这个 mask。

返回：

- `per_token_policy_gradient_loss`：shape `(batch_size, sequence_length)`；
- `metadata`：底层 loss 的统计信息，例如 clip-fraction component。

测试：

```
uv run pytest -k test_compute_policy_gradient_loss_on_policy
```

最后实现 token 与 sequence 两个维度上的 loss aggregation。

标准 GRPO 会：

1. 先对每条 sequence 的 response token 取平均；
2. 再对 batch 中的 sequences 取平均。

后面还会实现另一种做法：不按 sequence length 平均，而直接用固定常数归一化。后者更接近最初想估计的 policy gradient。

题目 (aggregate_loss_across_microbatch_sequence)：跨 token 与 sequence 聚合 loss (0.5 分)

实现 `aggregate_loss_across_microbatch`: 输入逐 token policy-gradient loss 和 response mask, 输出标量 average loss。

当前只需支持:

```
loss_normalization = "sequence"
```

推荐接口:

```
def aggregate_loss_across_microbatch(
    per_token_policy_gradient_loss: torch.Tensor,
    mask: torch.Tensor,
    loss_normalization: Literal["sequence", "constant"] = "sequence",
    normalization_constant: int | None = None,
) -> torch.Tensor:
```

参数:

- `per_token_policy_gradient_loss`: shape (batch_size, sequence_length);
- `mask`: shape (batch_size, sequence_length), 表示哪些位置参与 loss;
- `loss_normalization`:
 - "sequence": 每条 sequence 内平均, 再跨 sequences 平均;
 - "constant": 总 loss 除以一个固定常数;
- `normalization_constant`: 当使用 "constant" 时必需。

返回标量 `loss`, 并确保后续可以对其调用 `backward()`。

测试:

```
uv run pytest -k test_aggregate_loss_across_microbatch_sequence
```

4.2.4 GRPO training step

现在可以把前面的组件组合成一次完整的训练更新。

Gradient accumulation

为了让 inference 时 GPU 利用率较高, 需要使用较大的 rollout batch。On-policy RL 中, train batch size 等于 inference batch size, 因此训练 GPU 往往无法一次对整个 batch 求 gradient。

解决方法是把 batch 拆成多个 **microbatch**, 并在多个 microbatch 上进行 **gradient accumulation**。

关键难点在于正确处理 normalization, 使 microbatch 累积得到的 gradient 与直接对整个 batch 计算的 gradient 等价。

回顾 PyTorch: 每个 weight tensor 有一个 `.grad` 字段。

- 调用 `loss.backward()` 之前, 通常 `.grad is None`;
- 调用后, `.grad` 中保存 gradient;
- 通常 `optimizer.step()` 更新参数后, 用 `optimizer.zero_grad()` 清空 gradient。

普通训练:

```

# Forward pass.
logits = model(inputs)
loss = loss_fn(logits, labels)

# Backward pass.
loss.backward()

# Update weights.
optimizer.step()

# Zero gradients in preparation for next iteration.
optimizer.zero_grad()

```

要做 gradient accumulation, 可以把 batch 分成 k 个 microbatch, 只在所有 microbatch 都 backward 完成后调用一次 `optimizer.step()` 和一次 `optimizer.zero_grad()`。

对于 sequence normalization, microbatch average loss 还要按 microbatch 中 sequence 数量相对于整个 batch 的比例重新加权:

```

gradient_accumulation_steps = 4
microbatch_size = len(inputs) // gradient_accumulation_steps

for i in range(0, len(inputs), microbatch_size):
    inputs_microbatch = inputs[i:i + microbatch_size]
    labels_microbatch = labels[i:i + microbatch_size]

    # Forward pass.
    logits = model(inputs_microbatch)
    loss = loss_fn(logits, labels_microbatch) * (
        len(inputs_microbatch) / len(inputs)
    )

    # Backward pass.
    loss.backward()

# Update weights once across entire batch.
optimizer.step()

# Zero gradients once across entire batch.
optimizer.zero_grad()

```

后面使用 constant normalization 时, 由于 normalization 方式不同, microbatch 代码也需要稍作调整。

实现 train step

接下来实现一次 GRPO train step。函数接收 model、tokenizer、optimizer、reward function、prompts、rollouts 以及各类 hyperparameters, 并完成:

1. gradient accumulation;
2. backward;
3. 在 optimizer step 前, 把 gradient norm clip 到 `max_grad_norm`;
4. optimizer step;
5. 返回 batch training loss 与 metadata, 供 logging 使用。

至少记录以下指标:

- loss;
- gradient norm;
- token entropy;
- train rewards: total、format。

也可以记录其他指标。例如 `pass@k`：对 `Error: '_' allowed only in math mode`，统计每个 prompt 的前 k 个 rollout 中至少有一个正确答案的比例。

题目 (grpo_train_step_standard_on_policy)：GRPO train step (5 分)

实现给定 model、tokenizer 和 rollouts 的单个 policy-gradient batch update。

本阶段只需要支持标准 **on-policy GRPO**：

```
baseline = "mean"
advantage_normalizer = "std"
importance_reweighting_method = "none"
loss_normalization = "sequence"
```

对于其他输入可以 `raise NotImplementedError`。

推荐接口：

```
def grpo_train_step(
    model: PreTrainedModel,
    tokenizer: PreTrainedTokenizer,
    optimizer: Optimizer,
    gradient_accumulation_steps: int,
    max_grad_norm: float | None,
    reward_fn: Callable[[str, str], dict[str, float]],
    repeated_prompts: list[str],
    rollout_responses: list[str],
    repeated_ground_truths: list[str],
    group_size: int,
    # Reward normalization
    baseline: Literal["mean", "none"] = "mean",
    advantage_eps: float = 1e-6,
    advantage_normalizer: Literal["std", "none", "mean"] = "std",
    # Importance reweighting and clipping
    importance_reweighting_method: Literal["none", "noclip", "grpo", "gspo"] = "none",
    old_log_probs: torch.Tensor | None = None,
    cliprange: float | None = None,
    # Loss normalization
    loss_normalization: Literal["sequence", "constant"] = "sequence",
    normalization_constant: int | None = None,
) -> tuple[torch.Tensor, dict[str, torch.Tensor | float]]:
```

函数应使用 `gradient_accumulation_steps` 个 microbatch 执行 forward / backward。

参数：

- `model`：待训练的 Hugging Face model；
- `tokenizer`：用于 tokenization；
- `optimizer`：模型 optimizer；
- `gradient_accumulation_steps`：每次 optimizer step 中的 microbatch 数；

- `max_grad_norm`: 若不为 `None`, 在 `optimizer.step()` 前把 gradient norm clip 到该值;
- `reward_fn`: 给 response 与 ground truth 打分, 返回 `reward`、`format_reward`、`answer_reward`;
- `repeated_prompts`: 每个 prompt 因为有 `group_size` 个 rollout 而重复后的 prompt 列表, 长度为 `rollout_batch_size`;
- `rollout_responses`: policy 的 rollouts, 长度为 `rollout_batch_size = n_prompts_per_rollout_batch * group_size`;
- `repeated_ground_truths`: 与 prompt 同样重复后的 ground truth;
- `group_size`: 每个 question 的 response 数;
- `baseline`: "mean" 表示减去组内均值, "none" 表示不减;
- `advantage_eps`: 避免除零;
- `advantage_normalizer`: "std" / "none" / "mean";
- `importance_reweighting_method`: "none" / "noclip" / "grpo" / "gsdp";
- `old_log_probs`: 非 "none" 时需要;
- `cliprange`: GRPO/GSPO clipping 参数 ϵ ;
- `loss_normalization`: "sequence" 或 "constant";
- `normalization_constant`: 当使用 constant normalization 时需要。

返回:

- `loss`: 标量 tensor, 已经正确考虑 gradient accumulation, 可用于 logging;
- `metadata`: 至少包含底层 loss metadata、clip 前的 gradient norm, 以及其他希望记录的统计量。

测试:

```
uv run pytest -k test_grpo_train_step_standard_on_policy
```

4.3 实验

现在可以构造完整的 GRPO training loop。脚本应完成:

- 加载 model 与 dataset;
- 初始化 logging (例如 Weights & Biases)、vLLM server 和 optimizer;
- 运行 RL training loop。

推荐的 hyperparameters:

```
n_train_examples = 6400
n_val_examples = 1024
num_rollout_steps = 200
learning_rate = 1e-5
rollout_batch_size = train_batch_size = 256
group_size = 8
gradient_accumulation_steps = 32
sampling_temperature = 1.0
sampling_max_tokens = 512
max_grad_norm = 1.0

optimizer = torch.optim.AdamW(
    policy.parameters(),
    lr=learning_rate,
```

```
betas=(0.9, 0.95),
weight_decay=0.0,
)
```

注意，`rollout_batch_size` 和 `train_batch_size` 统计的是 **responses** 的数量，而不是 **prompts** 的数量。

所以：

```
rollout_batch_size = train_batch_size = 256
```

表示每个 batch 包含 32 个 prompts，每个 prompt 有 8 个 rollouts：

```
32 × 8 = 256 responses
```

一个合理的默认做法是每 10 个 rollout batch 在 validation set 上评估一次。至少使用 1024 个 validation examples，因为 CoT / RL evaluation 的噪声可能较大。

除了记录前面介绍的 metrics，也建议定期保存模型 rollouts，以便定性观察模型到底在做什么。一个合理默认值是每 40 个 rollout batch 记录一次当前 batch 的 training rollouts。

可以使用：

- `data/gsm8k/train.jsonl` 作为 training examples；
- `data/gsm8k/test.jsonl` 作为 validation examples。

由于 policy-gradient estimator 本身方差较高，而且 RL 具有 self-reinforcing 特性，不同随机 seed 的训练轨迹往往差异很大。因此，当确认代码正确后，需要用 **4 个 random seeds** 运行 RL loop。

固定 seed 下不完全可复现也没有关系，也可以直接独立运行 4 次。

画图时应显式体现不同 run 之间的 variance，而不是只画平均值。可选方案包括：

- 画 4 次 run 的平均值，同时也把每条实际曲线画出来；为了可读性，可以把单次 run 放到另一张图、降低透明度、减小线宽或改成虚线；
- 画平均值与 shaded confidence interval。每一步计算 sample mean 与 sample standard deviation，可使用讲义给出的区间：

$$\mu \pm 1.96 \sqrt{\frac{\sigma}{n}}, \quad (27)$$

其中 n 是 seed 数量；

- 画平均值，并同时画出每一步的 min 与 max。

题目 (`grpо_experiments_standard_on_policy`)：使用 GRPO 提升 OLMo-2-0425-1B 在 GSM8K 上的性能（约 2 B200 小时，10 分）

(a) 编写脚本，根据以下输入运行 GRPO training loop：

- model name；
- prompt；
- training set 文件路径；
- validation set 文件路径；
- sampling hyperparameters；
- training hyperparameters。

宏观流程应为：先初始化 vLLM server、WandB logging、datasets、model 和 optimizer；然后循环执行：

1. 把最新权重同步到 vLLM server；
2. 生成 training rollouts；
3. 在 training batch 上执行 policy-gradient update；
4. 定期在 validation set 上检查 performance，同时记录 generations。

交付内容： 在 OLMo-2-0425-1B + GSM8K 上运行标准 on-policy GRPO 的脚本。

(b) 先运行大约 50 steps，确认 validation reward 随训练提高，并且 rollout 看起来合理。

注意：reward 一开始接近 0 是正常的；根据 random seed，不同 run 可能需要若干步以后才第一次采样到 non-zero reward rollout，随后模型才开始明显改善。

交付内容： 能证明脚本正确的证据，例如 validation reward 持续提高、rollout 合理等。

(c) 确认脚本正确后，使用上面的 hyperparameters，对 OLMo-2-0425-1B 在 GSM8K 上运行 **4 个 random seeds**。使用 zero-shot `r1_zero` prompt。

若代码正确，应观察到 validation reward 随训练提高。

记录并绘制下列指标随时间的变化，同时体现不同 run 的 variance：

- loss；
- gradient norm；
- token entropy；
- train rewards：total、format；
- val rewards：total、format；
- val average response length；
- 其他任何你认为有助于 debugging 的指标。

还应定期记录 rollouts，并观察 response 是否因为训练而改善。

交付内容：

- 对每个指标给出图和几句分析，说明它如何随训练变化，以及不同 run 之间的 variance 有多大；
- 给出模型训练前与训练后的若干 rollout 示例；
- 给出一个过程，使最终 validation accuracy 在不同 random seeds 上平均至少达到 **25%**。

完成标准 GRPO 后，可以开始调 hyperparameter 与做 ablation。首先调整通常最影响训练效果的 learning rate。

题目 (grpo_learning_rate)：调节 learning rate (约 4 B200 小时，3 分)

以推荐 hyperparameters 为起点，对 learning rate 做 sweep：

- 至少测试一个低于推荐默认值的 learning rate；
- 至少测试一个高于推荐默认值的 learning rate。

报告最终 validation reward；如果 optimizer 发散，则说明发生 divergence。

根据上一部分观察到的 variance，自行决定每个 learning rate 需要运行多少 random seeds。

后续作业可以继续使用你调好的 learning rate，而不必固定使用推荐默认值。

交付内容： final validation reward 与 learning rate 的关系图，以及几句评论。

Prompt 同样会影响 RL dynamics，因为 RL 会强化模型自己探索出来的 rollout，而 prompt 会决定初始探索空间。下面研究 prompt choice 的影响。

题目 (grpo_prompt_ablation) : Prompt ablation (约 4 B200 小时, 3 分)

分别使用：

- `question_only` prompt;
- `r1_zero_three_shot` prompt;

运行 GRPO，并使用若干 random seeds。

与前面 zero-shot `r1_zero` prompt 的结果比较：

- 哪些 prompt 的 average reward 更高？
- 哪些 prompt 的 variance 更低？
- 其他 logged metrics 是否存在系统性差异？
- 考虑 run-to-run variance 后，你对这些结论有多大信心？

交付内容：评论，并附支持所述结论的图。

5 RL 算法变体

GRPO 是语言模型强化学习中常用的方法，但它里面的一些算法设计选择在不同论文中仍存在争议。本节将学习这些选择背后的部分理论论据，并通过受控实验亲自比较不同算法——至少在本作业所使用的模型与数据集组合上——的表现。

本节主要研究 **on-policy** 情形；下一节将进一步研究 **off-policy** 算法。

5.1 Dr. GRPO

在前面对 GRPO 的推导中，我们做了若干修改，使得最终的 GRPO policy-gradient estimator 不再严格具有原始目标 $\nabla_{\theta} J_{\theta}$ 的“正确期望”，其中 J_{θ} 是模型的 expected reward。

其中两个关键选择是：

1. 对 advantage 使用标准差归一化 (standard deviation advantage normalization) ；
2. 对每条 response 按 sequence length 进行归一化。

Dr. GRPO 论文 [Z. Liu et al., 2025] 主张撤销这两个选择：

- 不再用 group standard deviation 对 advantage 归一化；
- 不再按每条 sequence 自身长度进行归一化，而是用一个固定常数对总 loss 归一化。

把这个固定归一化常数记为 Z ，Dr. GRPO 的 gradient estimator 为：

$$\hat{g} \leftarrow \frac{1}{Z} \sum_{i=1}^B \sum_{j=1}^G \sum_{t=1}^{\text{len}(y^{(i,j)})} \left(r(y^{(i,j)} | x^{(i)}) - \mu_i \right) \nabla_{\theta} \log \pi_{\theta}(y_t^{(i,j)} | x^{(i)}, y_{<t}^{(i,j)}). \quad (33)$$

通常把 Z 设置为训练 batch size 乘以最大生成长度，即

$$Z = BGL, \quad (28)$$

其中 L 为最大 generation length；本作业中 $L = 512$ 。

题目 (think_about_length_normalization) : 思考 length normalization (1 分)

在运行任何实验之前，思考以下两种做法之间的区别：

- 每条 sequence 按自身 sequence length 进行归一化；
- 所有 sequence 都使用同一个固定常数进行归一化。

两种做法分别有什么优点和缺点？是否存在某些具体场景或例子，其中一种方式明显更合理？

交付内容：几句话的讨论。

题目 (`compute_group_normalized_rewards_drgrp`) : Dr. GRPO group normalization (0.5 分)

更新你的 `compute_group_normalized_rewards` 方法，使其支持：

```
advantage_normalizer = "none"
```

Dr. GRPO 本身使用：

```
baseline = "mean"
```

但在本题中，也请加入对：

```
baseline = "none"
```

的支持。

为了测试代码，实现 `adapters.run_compute_group_normalized_rewards`，然后运行：

```
uv run pytest -k compute_group_normalized_rewards_drgrp
```

并确认测试通过。

题目 (`aggregate_loss_across_microbatch_constant`) : Dr. GRPO loss aggregation (0.5 分)

更新你的 `aggregate_loss_across_microbatch` 方法，使其支持：

```
loss_normalization = "constant"
```

为了测试代码，实现 `adapters.run_aggregate_loss_across_microbatch`，然后运行：

```
uv run pytest -k test_aggregate_loss_across_microbatch_constant
```

并确认测试通过。

5.2 Rejection Fine-Tuning

接下来要研究的是一种简单得多的算法，通常称为 **rejection fine-tuning (RFT)** 或 **expert iteration (EI)**。

顾名思义，它的流程是：

1. 从模型中采样大量 rollouts；
2. 只保留其中正确的 rollouts；
3. 在这些正确 rollouts 上执行 supervised fine-tuning，也就是最小化 next-token prediction 的 log loss。

具体而言，RFT gradient 为：

$$\hat{g} \leftarrow \nabla_{\theta} \left[\frac{1}{Z} \sum_i \sum_{j=1}^G \mathbf{1}\{r(y^{(i,j)} | x^{(i)}) = 1\} \sum_{t=1}^{\text{len}(y^{(i,j)})} \log \pi_{\theta}(y_t^{(i,j)} | x^{(i)}, y_{<t}^{(i,j)}) \right]. \quad (34)$$

其中：

- Z 是固定的 normalizer，与 Dr. GRPO 中类似；
- $\mathbf{1}\{r(\dots) = 1\}$ 是 indicator function，只保留 reward 为 1 的正确回答。

下面的题目会让你思考：RFT gradient 是否真的是一个 policy gradient，也就是说，它是否真的在优化 expected reward J_{θ} ；同时还会比较它与 GRPO 的关系。

与前面一样，在 PyTorch 中实现时应返回该 objective 的负值，这样 optimizer 的 gradient descent 才等价于对原 objective 做 gradient ascent。

题目 (think_about_rft)：思考 RFT (2 分)

RFT objective (使用 constant normalization) 为：

$$J_{\theta} = \frac{1}{Z} \sum_x \sum_{j=1}^G \mathbf{1}\{r(y^{(j)} | x) = 1\} \log \pi_{\theta}(y^{(j)} | x), \quad (35)$$

其中：

- x 是 prompt；
- 每个 $y^{(j)}$ 都是从 $\pi_{\theta}(\cdot | x)$ 独立采样得到的 response；
- G 是每个 prompt 采样的 generations 数；
- Z 是固定归一化常数；
- r 是 reward function。

RFT gradient 就是 $\nabla_{\theta} J_{\theta}$ 。

相比之下，on-policy Dr. GRPO 的 policy-gradient estimator，即使用 constant normalization 且不进行 std normalization 的 GRPO，为：

$$\frac{1}{Z} \sum_x \sum_{j=1}^G \left(r(y^{(j)} | x) - \mu \right) \nabla_{\theta} \log \pi_{\theta}(y^{(j)} | x), \quad (36)$$

其中 group mean 为：

$$\mu = \frac{1}{G} \sum_{j=1}^G r(y^{(j)} | x). \quad (29)$$

假设 reward function r 是 binary 的，请比较这两个 objective：

- 它们的 expectation 是否相同？
- 你预计哪个 estimator 的 variance 更低？
- 从直觉上看，在什么情况下你可能更倾向使用其中一种？

交付内容：几句话的讨论。

5.3 MaxRL

最后，我们研究一种较新的方法：**Maximum Likelihood Reinforcement Learning (MaxRL)** [F. Tajwar et al., 2026]。

与标准 GRPO 用 group std 对 advantage 归一化、Dr. GRPO 完全不做 advantage normalization 不同，MaxRL 提议使用 **group mean** 进行归一化。

其 gradient estimator 为：

$$\hat{g} \leftarrow \frac{1}{Z} \sum_{i=1}^B \sum_{j=1}^G \sum_{t=1}^{\text{len}(y^{(i,j)})} \frac{r(y^{(i,j)} | x^{(i)}) - \mu_i}{\mu_i} \nabla_{\theta} \log \pi_{\theta}(y_t^{(i,j)} | x^{(i)}, y_{<t}^{(i,j)}). \quad (37)$$

原始 MaxRL 论文没有使用固定常数 Z ，而是除以 batch 中实际 token 总数：

$$\sum_{i=1}^B \sum_{j=1}^G \text{len}(y^{(i,j)}). \quad (30)$$

这个分母会随 batch 改变，而不是一个常数。

为了减少相对于其他 baseline 的变量、更加容易单独观察“除以 μ_i 而不是除以 std_i ”这一选择的影响，本作业仍使用 constant normalizer。

接下来的题目将用一些基础数学分析不同 advantage normalizer 的影响。

直觉上，用平均 reward μ_i 做除数，会平均意义上提高**困难 prompt**的权重，因为困难问题通常具有更低的 μ_i 。

我们可以把这一点形式化为对 expected reward 进行重新加权：

$$J_{\theta,w} = \mathbb{E}_{x \sim \rho} [w(x, \text{stopgrad}(\pi_{\theta})) \mathbb{E}_{y \sim \pi_{\theta}} r(y | x)], \quad (38)$$

其中 $w(x, \text{stopgrad}(\pi_{\theta}))$ 对 prompt x 进行重新加权。例如，可以根据当前 policy 在该 prompt 上的 expected reward：

$$\eta(x) = \mathbb{E}_{y \sim \pi_{\theta}} r(y | x) \quad (31)$$

来表示问题难度。

`stopgrad` 表示我们不对 **reweighting function** 本身求梯度。

下面将推导前面各种 advantage normalization 方法所隐式对应的 difficulty reweighting。

题目 (derive_difficulty_reweightings)：推导不同 advantage normalizer 所引入的 difficulty reweighting (6 分)

标准 policy gradient 在 prompt 分布上的整体形式为：

$$\nabla_{\theta} J_{\theta} = \nabla_{\theta} \mathbb{E}_{x \sim \rho} [\mathbb{E}_{y \sim \pi_{\theta}} r(y | x)]. \quad (39)$$

这里：

- ρ 是 prompt distribution；
- π_{θ} 是 policy；
- $r(y | x)$ 表示 response y 对 prompt x 是否正确。

本题要推导不同 GRPO 变体实际优化的 surrogate objective，它们都写成：

$$\nabla_{\theta} J_{\theta,w} = \nabla_{\theta} \mathbb{E}_{x \sim \rho} [w(x, \text{stopgrad}(\pi_{\theta})) \mathbb{E}_{y \sim \pi_{\theta}} r(y | x)]. \quad (40)$$

其中 w 是对 prompts 的 reweighting，并且不对 w 求梯度。

(a) Dr. GRPO

Dr. GRPO policy-gradient estimator 为：

$$\mathbb{E}_{x \sim \rho} \left[\frac{1}{Z} \sum_{j=1}^G (r(y^{(j)} | x) - \mu) \nabla_{\theta} \log \pi_{\theta}(y^{(j)} | x) \right], \quad (41)$$

其中：

$$\mu = \frac{1}{G} \sum_{j=1}^G r(y^{(j)} | x). \quad (32)$$

令固定 normalizer $Z = G$ ，并令 group size $G \rightarrow \infty$ 。此时，这个 estimator 等价于优化哪个 reweighting function w 下的 surrogate objective $J_{\theta, w}$ ？

交付内容： 用题目参数的某个子集写出表达式，并给出几句推导或解释。

(b) GRPO

假设同样使用 constant normalization，标准 GRPO 与 Dr. GRPO 的不同之处是，它还除以 group standard deviation：

$$\mathbb{E}_{x \sim \rho} \left[\frac{1}{Z} \sum_{j=1}^G \frac{r(y^{(j)} | x) - \mu}{\text{std}} \nabla_{\theta} \log \pi_{\theta}(y^{(j)} | x) \right]. \quad (42)$$

令 $Z = G$ ，并令 $G \rightarrow \infty$ 。此时对应哪个 reweighting function w ？

交付内容： 一个表达式，以及几句解释。

(c) MaxRL

MaxRL 改为除以 group mean：

$$\mathbb{E}_{x \sim \rho} \left[\frac{1}{Z} \sum_{j=1}^G \frac{r(y^{(j)} | x) - \mu}{\mu} \nabla_{\theta} \log \pi_{\theta}(y^{(j)} | x) \right]. \quad (43)$$

令 $Z = G$ ，并令 $G \rightarrow \infty$ 。此时对应哪个 reweighting function w ？

交付内容： 一个表达式，以及几句解释。

题目 (think_about_advantage_normalization)：思考 advantage normalization (2 分)

在运行任何实验之前，思考以下三种方法的区别：

- 用 group std 对 group advantages 归一化；
- 用 group mean 归一化；
- 完全不做 advantage normalization。

它们分别有什么优点与缺点？是否存在某些具体设置或例子，其中某一种明显更合理？

交付内容： 几句话的讨论。

题目 (compute_group_normalized_rewards_maxrl)：MaxRL group normalization (0.5 分)

更新 `compute_group_normalized_rewards`，使其支持：

```
advantage_normalizer = "mean"
```

和 `advantage_normalizer = "std"` 一样，请在 normalizer 上加入 `advantage_eps`，避免除以 0。

为了测试代码，实现 `adapters.run_compute_group_normalized_rewards`，然后运行：

```
uv run pytest -k compute_group_normalized_rewards_maxrl
```

并确认测试通过。

5.4 实验

现在可以用实验比较这些不同的 policy-gradient estimators。

首先，需要更新 `grpo_train_step`，支持前面实现的各种变体。具体配置如下：

- `GRPO_constant` :
 - `baseline = "mean"`
 - `advantage_normalizer = "std"`
 - `loss_normalization = "constant"`
- `Dr_GRPO` :
 - `baseline = "mean"`
 - `advantage_normalizer = "none"`
 - `loss_normalization = "constant"`
- `RFT` :
 - `baseline = "none"`
 - `advantage_normalizer = "none"`
 - `loss_normalization = "constant"`
- `MaxRL` (constant normalization 版本) :
 - `baseline = "mean"`
 - `advantage_normalizer = "mean"`
 - `loss_normalization = "constant"`

为了加速训练，注意：一旦已经为每条 sequence 计算出了 normalized advantage，那么 **advantage 为 0 的 sequence 不必再送入模型**，因为它们对 gradient 的贡献就是 0。

由于本作业 reward 是 binary 的，sequence 在以下两种情况下会有 zero advantage：

- 若 `baseline = "mean"`，当同一 group 中所有 sequence 的 reward 都相同时，该 group 中所有 sequence 的 advantage 都为 0；
- 若 `baseline = "none"`，所有 reward 为 0 的 sequence，其 advantage 都为 0。

剪掉 zero-advantage sequences 后，也可以按相同比例减少 `gradient_accumulation_steps`，从而保持 `microbatch_size` 不变。

例如，如果一半 sequences 的 advantage 为 0，那么可以把 grad accumulation steps 从 k 减少到 $k/2$ 。

这种优化对 RFT 尤其有用，因为 RFT 中会存在大量 zero-reward sequences。

不过必须仔细处理数学和实现，确保“剪枝版”所计算的 gradient 与“不剪枝版”相同。

题目 (`grpo_train_step_variants_on_policy`)：GRPO train-step 变体 (2.5 分)

更新 `grpo_train_step`，使其支持完整的 on-policy 变体范围，同时仍保持：

```
importance_reweighting_method = "none"
```

需要支持：

```
baseline: Literal["mean", "none"]
advantage_normalizer: Literal["std", "none", "mean"]
loss_normalization: Literal["sequence", "constant"]
```

此外，为了加速训练，你的方法应避免把 zero-advantage sequences 传入模型。

特别注意：当 `baseline = "none"` 时，错误样本 reward 为 0，因此它们在 loss 中的权重也是 0，没有必要把它们送入训练模型。可以利用这一点加速训练。

为了测试代码，实现 `adapters.run_grpo_train_step`，然后运行：

```
uv run pytest -k test_grpo_train_step_variants_on_policy
```

并确认测试通过。

现在 train step 已经实现，可以运行实验比较这些方法的性能，并判断它们是否优于标准 GRPO。为了保证可比性，请使用与之前 runs 相同的 hyperparameters。

关于 hyperparameter tuning 的说明：

如果目标是严格判断一种方法是否优于另一种方法，最合理的实验方式是分别为每种方法单独调 hyperparameters。但这样计算成本很高，本作业没有足够算力。

一个更便宜的方法是：只为 baseline 调 hyperparameters，然后把同一组参数用于新方法。例如，本作业先在标准 GRPO 上调 learning rate，再把这个 learning rate 用于后续方法。

在这种情况下：

- 如果新方法在并未专门调参的情况下仍优于 baseline，那么可以合理地认为它更好，因为未调好的 learning rate 只会给新方法的真正最优性能提供一个 lower bound；
- 反过来，如果新方法表现更差，则不能据此断言它确实差于 baseline，因为它自己的超参数还没有被充分调优。

题目 (grpo_experiments_variants_on_policy)：比较不同 RL 算法的性能（约 8 B200 小时，10 分）

保持 hyperparameters 与之前标准 GRPO runs 一致——learning rate 使用你之前选择的值——分别运行以下变体，每种方法运行 **4 个 random seeds**。使用 zero-shot `rl_zero` prompt。

- `GRPO_constant`：标准 GRPO，但 loss aggregation 使用 constant normalization，而不是 sequence normalization。
- `Dr_GRPO`：在 `GRPO_constant` 基础上设置 `advantage_normalizer = "none"`。
- `RFT`：在 `GRPO_constant` 基础上设置 `advantage_normalizer = "none"` 且 `baseline = "none"`。对于该方法，无需把错误样本送入训练模型，因此应该更快。
- `MaxRL`：在 `GRPO_constant` 基础上设置 `advantage_normalizer = "mean"`。原始 MaxRL 实现按每个 microbatch 的 token 数归一化 loss，但本作业使用 constant normalization。

与之前标准 GRPO runs 相比：

- 这些方法表现如何？
- 哪种方法 average performance 最好？
- 哪种方法 variance 最低？
- 哪些方法可能会明显受益于进一步调 hyperparameters？

- 根据不同 runs 之间的 variance，你对这些观察结果有多大信心？

交付内容：评论，并附支持所述结论的图。

6 Off-policy RL

到目前为止，我们的算法和实验都完全是 **on-policy**：每个 gradient estimator 使用的样本都直接来自当前正在更新的模型。换句话说，每个 inference batch 只执行 1 个 training step，并且 training batch size 与 inference batch size 相等。

本节将研究 **off-policy RL**：每个 inference batch 执行多个 training steps。这样有机会显著加快训练，但代价是潜在的不稳定性和更高的算法复杂度。

6.1 Importance reweighting

在最基本的 policy gradient 中，我们先采样一个 response batch，然后在整个 batch 上执行一次 large-batch gradient update。

一种自然的想法是：把这个 batch 拆成多个 minibatches，每个 minibatch 都执行一次 training step，也许模型就能训练得更快。

这种方法称为 **off-policy RL**。与之相对，原来的“一个 inference batch 对应一个 training step”的方式叫 on-policy RL。

之所以称为 off-policy，是因为执行第一个 minibatch update 后，当前 policy 已经发生变化，不再等于当初用来生成 rollouts 的 policy。

因此剩余样本就变成了 **off-policy** 或 **stale samples**：它们并不是从当前 policy 采样出来的。

记 inference 时使用的旧 policy 为 π_0 ，当前 policy 为 π_θ ，则：

$$\mathbb{E}_{x \sim \rho} \mathbb{E}_{y \sim \pi_0(y|x)} [r(y|x) \nabla_\theta \log \pi_\theta(y|x)] \neq \mathbb{E}_{x \sim \rho} \mathbb{E}_{y \sim \pi_\theta(y|x)} [r(y|x) \nabla_\theta \log \pi_\theta(y|x)]. \quad (44)$$

左侧是一个“naive” off-policy estimator：我们假装 response y 来自当前 policy，然后机械地按 on-policy 方法算 gradient。

上式说明，这个 naive estimator 的 expectation 并不正确。

一种修正 bias 的方法是 **importance reweighting**：

$$\mathbb{E}_{x \sim \rho} \mathbb{E}_{y \sim \pi_0(y|x)} \left[\frac{\pi_\theta(y|x)}{\pi_0(y|x)} r(y|x) \nabla_\theta \log \pi_\theta(y|x) \right]. \quad (45)$$

importance weight 为：

$$\frac{\pi_\theta(y|x)}{\pi_0(y|x)}. \quad (33)$$

它会提高那些在当前 policy π_θ 下仍然很可能出现的 response 的权重，同时降低在当前 policy 下概率已经很低的 stale samples 的权重。

可以直接验证，这个修正后的 expression 具有正确 expectation：

$$\mathbb{E}_{x \sim \rho} \mathbb{E}_{y \sim \pi_0} \left[\frac{\pi_\theta(y|x)}{\pi_0(y|x)} r(y|x) \nabla_\theta \log \pi_\theta(y|x) \right] = \mathbb{E}_{x \sim \rho} \mathbb{E}_{y \sim \pi_\theta} [r(y|x) \nabla_\theta \log \pi_\theta(y|x)]. \quad (46)$$

不过 importance reweighting 的缺点是会增加 estimator 的 variance。

如果 importance weight 最大可能达到 C ，variance 也可能放大约 C 倍。

对于语言模型，这尤其严重，因为 sequence-level importance weight 是每个 token importance ratio 的乘积：

$$\frac{\pi_\theta(y|x)}{\pi_0(y|x)} = \prod_{t=1}^{\text{len}(y)} \frac{\pi_\theta(y_t|x, y_{<t})}{\pi_0(y_t|x, y_{<t})}. \quad (47)$$

它会随 response length 呈指数式变化。

因此：

- naive、无 importance weighting 的 off-policy estimator：bias 很高，但没有额外 reweighting variance；
- 完整 sequence-level importance-reweighted estimator：bias 为 0，但 variance 很高。

接下来研究的方法会处于两者之间，用不同 heuristic 在 **bias-variance tradeoff** 中折中。

6.2 PPO/GRPO 风格的 importance reweighting 与 clipping

6.2.1 Token-level reweighting

标准方法最早由 PPO [J. Schulman et al., 2017] 提出，之后也被 GRPO [Z. Shao et al., 2024] 采用：不做 sequence-level importance reweighting，而改成 **token-level importance reweighting**。

这样得到未 clipping 的 token-level estimator：

$$\mathbb{E}_{x \sim \rho} \mathbb{E}_{y \sim \pi_0(y|x)} \left[r(y|x) \sum_{t=1}^{\text{len}(y)} \frac{\pi_\theta(y_t | x, y_{<t})}{\pi_0(y_t | x, y_{<t})} \nabla_\theta \log \pi_\theta(y_t | x, y_{<t}) \right]. \quad (48)$$

这与前面的“principled” sequence-level estimator 并不相同。后者展开时间步后，可以写成：

$$\mathbb{E}_{x \sim \rho} \mathbb{E}_{y \sim \pi_0(y|x)} \left[r(y|x) \left(\prod_{t=1}^{\text{len}(y)} \frac{\pi_\theta(y_t | x, y_{<t})}{\pi_0(y_t | x, y_{<t})} \right) \sum_{t=1}^{\text{len}(y)} \nabla_\theta \log \pi_\theta(y_t | x, y_{<t}) \right]. \quad (49)$$

token-level 方法最大的好处是：reweighting term 不再是 $\text{len}(y)$ 个 ratio 的乘积，因此 variance 大幅降低。

为了理解这个 estimator 实际在优化什么，可以推导它对应的 surrogate objective [T. Degris et al., 2013]。

定义 surrogate policy $\tilde{\pi}_t$ ：除了第 t 个 token 由当前 policy π_θ 采样以外，其余所有 token 都由旧 policy π_0 采样：

$$\tilde{\pi}_t(y|x) = \left(\prod_{s=1}^{t-1} \pi_0(y_s | x, y_{<s}) \right) \pi_\theta(y_t | x, y_{<t}) \left(\prod_{s=t+1}^{\text{len}(y)} \pi_0(y_s | x, y_{<s}) \right). \quad (50)$$

如果假设所有 response 长度都为 L ，token-level reweighting 实际优化的是这些 surrogate policies 下 expected reward 的总和：

$$J_\theta^{\text{token}} = \mathbb{E}_x \left[\sum_{t=1}^L \mathbb{E}_{y \sim \tilde{\pi}_t(y|x)} [r(y|x)] \right]. \quad (51)$$

这个结论可以通过 importance reweighting 直接得到：

$$\nabla_\theta J_\theta^{\text{token}} = \nabla_\theta \mathbb{E}_x \left[\sum_{t=1}^L \mathbb{E}_{y \sim \tilde{\pi}_t(y|x)} [r(y|x)] \right] \quad (52)$$

$$= \nabla_\theta \mathbb{E}_x \left[\sum_{t=1}^L \mathbb{E}_{y \sim \pi_0(y|x)} \left[\frac{\pi_\theta(y_t | x, y_{<t})}{\pi_0(y_t | x, y_{<t})} r(y|x) \right] \right] \quad (53)$$

$$= \mathbb{E}_x \left[\mathbb{E}_{y \sim \pi_0(y|x)} \left[\sum_{t=1}^L \frac{\pi_\theta(y_t | x, y_{<t})}{\pi_0(y_t | x, y_{<t})} r(y|x) \nabla_\theta \log \pi_\theta(y_t | x, y_{<t}) \right] \right]. \quad (54)$$

最后一步仍使用了前面反复使用的 log-derivative trick。

这个 surrogate objective 也说明了 token-level reweighting 引入的 bias：

1. 在第 t 项中，prefix $y_{<t}$ 来自旧 policy π_0 ，而不是当前 policy π_θ ，因此它可能不能代表当前模型真正会遇到的

prefix;

2. 采样完当前 token y_t 后, 后续 suffix $y_{>t}$ 仍来自旧 policy π_0 , 而不是当前 policy。也就是说, 目标函数在评估当前 action 对未来行为的影响时, 依据的是 stale policy 的后续行为。

完整 sequence-level reweighting 能同时修正 prefix 和 suffix 的权重, 但代价是更高 variance。

直觉上, 当前 policy π_θ 与旧 policy π_0 差距越大, 这些 bias 就越严重。

题目 (derive_surrogate_objectives) : 推导 importance reweighting 方法的 surrogate objective (2 分)

上面已经看到, token-level importance reweighting 等价于在一类 surrogate policies 下优化 expected reward, 其中只有一个位置来自当前 policy, 其余位置来自旧 policy。

现在考虑一种 **pairwise importance reweighting** 方法, 其 policy-gradient estimator 为:

$$\sum_{t=1}^{L/2} \frac{\pi_\theta(y_{2t-1} | x, y_{<2t-1}) \pi_\theta(y_{2t} | x, y_{<2t})}{\pi_0(y_{2t-1} | x, y_{<2t-1}) \pi_0(y_{2t} | x, y_{<2t})} r(y | x) \nabla_\theta [\log(\pi_\theta(y_{2t-1} | x, y_{<2t-1}) \pi_\theta(y_{2t} | x, y_{<2t}))]. \quad (55)$$

其中:

- π_θ 是当前 policy;
- π_0 是 stale sampling policy;
- $y \sim \pi_0(y | x)$ 。

这个 estimator 优化的 surrogate objective 是什么?

交付内容: 一个由题目参数组成的表达式, 并附推导。

6.2.2 Clipping

除了 token-level reweighting, PPO 和 GRPO 引入的另一个主要 heuristic 是 **importance-weight clipping**。

前面看到, 随着 π_θ 与 π_0 越来越远:

- importance reweighting 的 variance 会越来越严重;
- token-level surrogate objective 的 bias 也会越来越严重。

因此, 为了让 off-policy RL 保持稳定, 需要限制当前 policy 不要偏离旧 policy 太远。

最简单的方法是减少每个 inference batch 对应的 training steps 数, 但有些 batch 可能允许更多 update。为了尽可能提高效率, 希望在一个 inference batch 上多做一些训练, 同时避免 policy drift 过大。

因此, 一类常见方法就是对过大或过小的 importance weights 做 clipping。

不同论文对 clipping 的实现细节存在分歧。本作业实现 PPO 中的 clipping 方式, 该方式之后也延续到了 GRPO, 并且目前较为常见。

先将 token-level importance reweighting 与标准 GRPO objective 结合, 得到不 clipping 的 loss:

$$J_\theta^{\text{GRPO-off-policy-noclip}} = \frac{1}{BG} \sum_{i=1}^B \sum_{j=1}^G \frac{1}{\text{len}(y^{(i,j)})} \frac{r(y^{(i,j)} | x^{(i)}) - \mu_i}{\text{std}_i} \sum_{t=1}^{\text{len}(y^{(i,j)})} \frac{\pi_\theta(y_t | x, y_{<t})}{\pi_0(y_t | x, y_{<t})}. \quad (56)$$

这里 samples 来自 stale inference policy π_0 。对这个 objective 求导, 就得到前面推导的 token-reweighted policy-gradient estimator, 只是这里还包含 GRPO advantage 和 sequence normalization。

现在加入 clipping。定义:

$$A^{(i,j)} = \frac{r(y^{(i,j)} | x^{(i)}) - \mu_i}{\text{std}_i} \quad (34)$$

为 response j 在 prompt i 上的 advantage, 并定义第 t 个 importance ratio:

$$w_t^{(i,j)} = \frac{\pi_\theta(y_t | x, y_{<t})}{\pi_0(y_t | x, y_{<t})}. \quad (35)$$

clipped objective 为:

$$J_\theta^{\text{GRPO-off-policy-clip}} = \frac{1}{BG} \sum_{i=1}^B \sum_{j=1}^G \frac{1}{\text{len}(y^{(i,j)})} \sum_t \min \left(A^{(i,j)} w_t^{(i,j)}, A^{(i,j)} \text{clip}(w_t^{(i,j)}, [1 - \varepsilon, 1 + \varepsilon]) \right). \quad (57)$$

其中:

$$\text{clip}(w, [1 - \varepsilon, 1 + \varepsilon]) = \min(\max(w, 1 - \varepsilon), 1 + \varepsilon). \quad (36)$$

也就是把 importance weight 限制在 $[1 - \varepsilon, 1 + \varepsilon]$ 内。

一个更直观的等价写法是 [J. Achiam, 2018]:

$$J_\theta^{\text{GRPO-off-policy-clip}} = \frac{1}{BG} \sum_{i,j} \frac{1}{\text{len}(y^{(i,j)})} \sum_t \begin{cases} \min(w_t^{(i,j)}, 1 + \varepsilon) A^{(i,j)}, & A^{(i,j)} \geq 0, \\ \max(w_t^{(i,j)}, 1 - \varepsilon) A^{(i,j)}, & A^{(i,j)} < 0. \end{cases} \quad (58)$$

直观理解:

- 对 positive advantage action, 模型被鼓励提高其概率, 直到当前 policy 的概率相对旧 policy 提升到 $1 + \varepsilon$ 倍; 继续增加后, 该项进入 clipped branch, 梯度变为 0;
- 对 negative advantage action, 模型被鼓励降低其概率, 直到概率降到旧 policy 的 $1 - \varepsilon$ 倍; 再降低后梯度变为 0。

因此 gradient estimator 可以写为:

$$\nabla_\theta J_\theta^{\text{GRPO-off-policy-clip}} = \frac{1}{BG} \sum_{i,j} \frac{1}{\text{len}(y^{(i,j)})} \sum_t \text{mask}_t^{(i,j)} w_t^{(i,j)} A^{(i,j)} \nabla_\theta \log \pi_\theta(y_t | x, y_{<t}), \quad (59)$$

其中 mask 为:

$$\text{mask}_t^{(i,j)} = \begin{cases} \mathbf{1}\{w_t^{(i,j)} < 1 + \varepsilon\}, & A^{(i,j)} \geq 0, \\ \mathbf{1}\{w_t^{(i,j)} > 1 - \varepsilon\}, & A^{(i,j)} < 0. \end{cases} \quad (60)$$

也就是说:

- 对 positive advantage, 屏蔽过大的 importance weight;
- 对 negative advantage, 屏蔽过小的 importance weight。

现在已经具备实现:

```
importance_reweighting_method = "noclip"
```

和:

```
importance_reweighting_method = "grpo"
```

所需的数学基础。二者分别对应:

- $J_\theta^{\text{GRPO-off-policy-noclip}};$

- $J_{\theta}^{\text{GRPO-off-policy-clip}}$ 。

与 on-policy 情况一样，在实现 loss 时要返回 objective 的负值，使 gradient descent 实现目标所需的 gradient ascent。

题目 (compute_policy_gradient_loss_off_policy)：使用 token-level reweighting 的 off-policy policy gradient (1 分)

更新 `compute_policy_gradient_loss`，使其支持：

```
importance_reweighting_method = "noclip"
```

或：

```
importance_reweighting_method = "grpo"
```

需要使用：

- `old_log_probs`：生成 rollouts 时旧模型在每个 token 上的 log-probability；你需要在 training script 中额外添加几行代码计算这些值；
- `cliprange`：clipping strength 参数 ϵ 。

为了测试代码，实现 `adapters.run_compute_policy_gradient_loss`，然后运行：

```
uv run pytest -k test_compute_policy_gradient_loss_off_policy
```

并确认测试通过。

补充说明：

在 PPO 中，clipping 通常被解释为防止当前 policy 偏离旧 policy 太远。但也可以把它看成一种直接降低 importance-reweighted estimator variance 的方法，因为它压制了过大的 importance weights。

如果并不特别担心 policy 与旧 policy 相距太远，而更希望更新激进一些，那么还有一个更直接的办法：只对 gradient estimator 中的 importance weight 设置上界：

$$\hat{g} \leftarrow \frac{1}{BG} \sum_{i,j} \frac{1}{\text{len}(y^{(i,j)})} \sum_t \min(w_t^{(i,j)}, 1 + \epsilon) A^{(i,j)} \nabla_{\theta} \log \pi_{\theta}(y_t \mid x, y_{<t}). \quad (61)$$

这里与 PPO/GRPO 的区别是：即使某个 positive-advantage action 已经被当前 policy 提高了超过 $1 + \epsilon$ 倍，它仍然会保留非零 gradient，只是 importance weight 本身被截断。

这种更简单的 clipping 方法来自 **CISPO** [MiniMax et al., 2025]。原论文还会按 group 的 token count 进行 normalization，而不是按每条 sequence length；这里为了简化，仅写成 sequence-normalized objective。

该方法是可选内容，你也可以尝试 CISPO 并与其他 clipping 方法比较。

6.3 GSPO

上面已经看到 token-level reweighting 存在 bias，因为它忽略了 prefix 和 suffix 的完整 reweighting。

GSPO 论文 [C. Zheng et al., 2025] 认为 GRPO 可能不稳定，并主张回到 **sequence-level reweighting**。

但完整 sequence importance weight 是 L 个 ratio 的乘积，variance 很容易爆炸。GSPO 的处理方式是：对这个乘积取 $1/L$ 次方。

换句话说，它使用 sequence 上所有 token importance weights 的几何平均数 (**geometric mean**)，而不是乘积。

对应 loss 为：

$$J_{\theta}^{\text{GRPO-off-policy-gspo}} = \frac{1}{BG} \sum_{i=1}^B \sum_{j=1}^G \min \left(A^{(i,j)} s^{(i,j)}, A^{(i,j)} \text{clip}(s^{(i,j)}, [1 - \varepsilon, 1 + \varepsilon]) \right), \quad (62)$$

其中 sequence-level importance weight:

$$s^{(i,j)} = \left(\prod_{t=1}^{\text{len}(y^{(i,j)})} \frac{\pi_{\theta}(y_t \mid x, y_{<t})}{\pi_0(y_t \mid x, y_{<t})} \right)^{1/\text{len}(y^{(i,j)})}. \quad (63)$$

注意，如果去掉 $1/\text{len}(y)$ 次方和 clipping，这个 objective 就退化成完整的 sequence-level importance-reweighted policy-gradient loss。

取 geometric mean 再 clipping 的作用是：牺牲一些无偏性，以换取显著更低的 variance。

另外，geometric mean 本身在求导时会自然产生 sequence-length normalization。暂时忽略 clipping，有：

$$\nabla_{\theta} J_{\theta}^{\text{GRPO-off-policy-gspo}} = \frac{1}{BG} \sum_{i,j} A^{(i,j)} \nabla_{\theta} s^{(i,j)} \quad (64)$$

进一步得到：

$$= \frac{1}{BG} \sum_{i,j} A^{(i,j)} s^{(i,j)} \frac{1}{\text{len}(y^{(i,j)})} \sum_t \nabla_{\theta} \log \pi_{\theta}(y_t \mid x, y_{<t}). \quad (65)$$

因此，如果你想把 GSPO 应用于自己喜欢的 constant-normalized RL algorithm（可选），需要相应调整 sequence-level importance weight 的 exponent。

题目 (think_about_importance_reweighting)：思考 importance reweighting (2 分)

考虑 off-policy RL 中三种可能的 importance reweighting 策略：

1. 完全不使用 importance reweighting；
2. PPO/GRPO 风格的 clipped token-level importance reweighting；
3. GSPO 风格、使用 geometric mean 的 clipped sequence-level importance reweighting。

从 bias-variance tradeoff 的角度看，这三种方法分别位于什么位置？你能否想到一些场景，其中一种方法在直觉上比另外两种更合适？

交付内容：几句话的评论。

题目 (compute_policy_gradient_loss_off_policy_gspo)：使用 sequence-level reweighting 的 off-policy policy gradient (1 分)

更新 `compute_policy_gradient_loss`，使其支持：

```
importance_reweighting_method = "gspo"
```

该模式需要使用：

- `old_log_probs`：生成 rollout 时旧模型对各 token 的 log-probability；
- `cliprange`：clipping strength ε 。

和前面一样，你需要在 training script 中添加几行代码计算 `old_log_probs`。

实现时应以数值稳定的方式计算 geometric mean。

仍然返回 objective 的负值，使最小化 loss 等价于所需的 gradient ascent。

为了测试，实现 `adapters.run_compute_policy_gradient_loss`，然后运行：

```
uv run pytest -k test_compute_policy_gradient_loss_off_policy_gspo
```

并确认测试通过。

6.4 实验

现在已经有了更新 `grpo_train_step`、支持 off-policy training 所需的全部组件。

题目 (`grpo_train_step_off_policy`)：Off-policy GRPO train step (2.5 分)

更新 `grpo_train_step`，支持以下 off-policy 参数：

- `importance_reweighting_method`
- `old_log_probs`
- `cliprange`

此时该函数应支持前面定义的完整参数范围。

为了测试，实现 `adapters.run_grpo_train_step`，然后运行：

```
uv run pytest -k test_grpo_train_step_off_policy
```

并确认测试通过。

接下来通过实验研究：

- off-policy training 到底能加快多少？
- 稳定性会损失多少？
- 哪种 reweighting/clipping 方法更好？

除以下设置外，请保持其他 hyperparameters 与之前 RL runs 一致。现在使用 **32x off-policy**：每个 inference batch 执行 32 个 training steps。

```
rollout_batch_size = 256
train_batch_size = 8
gradient_accumulation_steps = 1
```

作为默认值，可使用原始论文中的 clipping range：

- `offpolicy_clip`: `cliprange = 0.2`
- `offpolicy_gspo`: `cliprange = 3e-4`

如果愿意，也可以自己调这些值。

此外，你还需要在 training script 中添加代码计算 `old_log_probs`，以便传入 reweighted loss。

题目 (`grpo_experiments_off_policy`)：比较不同 off-policy 算法的性能（约 8 B200 小时，10 分）

保持 hyperparameters 与标准 GRPO runs 相同——learning rate 使用你自己选择的值——运行以下变体，每种方法 **4 个 random seeds**。使用 zero-shot `r1_zero` prompt。

如果愿意，你可以使用上一节自己最喜欢的 RL algorithm variant，而不一定继续使用标准 GRPO。

- `offpolicy_naive`: 不再让 inference batch size 和 training batch size 都为 256, 而是把 training batch size 缩小到 inference batch size 的 $1/32$:

$$256/32 = 8. \quad (37)$$

同时把 gradient accumulation steps 按同样比例减少, 以保证 GPU 利用率。保持:

```
importance_reweighting_method = "none"
```

- `offpolicy_noclip`: 在 `offpolicy_naive` 基础上设置:

```
importance_reweighting_method = "noclip"
```

- `offpolicy_clip`: 在 `offpolicy_naive` 基础上设置:

```
importance_reweighting_method = "grpo"
```

- `offpolicy_gspo`: 在 `offpolicy_naive` 基础上设置:

```
importance_reweighting_method = "gspo"
```

除了之前已经记录的 metrics, 还务必记录 **clip fraction**。

与 fully on-policy GRPO 相比:

- 这些 off-policy 方法表现如何?
- 转成 off-policy 后, training stability 和不同 runs 间 variance 有何变化?
- 两种 clipping 方法在 clip fraction 上有什么不同? 哪一种看起来更稳定?
- 哪些方法可能明显受益于进一步 hyperparameter tuning?
- 根据 run-to-run variance, 你对观察结果有多大信心?

交付内容: 评论, 并附支持所述结论的图。

7 尝试你自己的 Policy-Gradient Estimator

题目 (try_your_own): 尝试你自己的 policy-gradient estimator (10 分)

到这里, 你已经接触了文献中提出的多种 policy-gradient estimators, 也学习了它们背后的一些理论。现在需要提出你自己的方法。

可以考虑:

- 尝试不同的 advantage estimator, 从而引入不同的 prompt reweighting;
- 尝试不同的 importance reweighting 策略;
- 在 advantage estimator 中使用不同的 reward baseline。

提出你自己的 policy-gradient estimator, 然后仍然在同一个 RL 设置上运行:

- 模型: OLMo-2-0425-1B;
- 数据集: GSM8K。

使用多个 random seeds, 并保持其他 problem parameters 固定, 使结果能够与之前的方法公平比较。

你的方法应当相对于前面某个已经尝试过的方法只修改一个因素。也就是说，至少应能找到一个本作业之前的方法，使你的方法与它相比只有一个变化。

回答：

- 你提出这一修改的理论依据或直觉是什么？
- 你的方法是否优于之前的 baselines？

交付内容：评论，并附相关指标图；如果数学推导有助于解释方法，也可以一并提供。

参考文献

为保持引用信息准确，参考文献题名保留原文：

1. J. Li et al., "DataComp-LM: In search of the next generation of training sets for language models." [Online]. Available: <https://arxiv.org/abs/2406.11794>
2. T. OLMo et al., "2 OLMo 2 Furious." [Online]. Available: <https://arxiv.org/abs/2501.00656>
3. K. Cobbe et al., "Training Verifiers to Solve Math Word Problems." [Online]. Available: <https://arxiv.org/abs/2110.14168>
4. DeepSeek-AI et al., "DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning." [Online]. Available: <https://arxiv.org/abs/2501.12948>
5. W. Kwon et al., "Efficient Memory Management for Large Language Model Serving with PagedAttention." 2023.
6. Z. Liu et al., "Understanding R1-Zero-Like Training: A Critical Perspective." [Online]. Available: <https://arxiv.org/abs/2503.20783>
7. OpenAI et al., "OpenAI o1 System Card." [Online]. Available: <https://arxiv.org/abs/2412.16720>
8. Z. Shao et al., "DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models." [Online]. Available: <https://arxiv.org/abs/2402.03300>
9. J. Achiam, "Spinning Up in Deep Reinforcement Learning," 2018.
10. N. Lambert, "Reinforcement Learning from Human Feedback." [Online]. Available: <https://rlhfbook.com/>
11. R. J. Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning," *Machine Learning*, vol. 8, no. 3-4, pp. 229-256, 1992, doi: 10.1007/BF00992696.
12. F. Tajwar et al., "Maximum Likelihood Reinforcement Learning." [Online]. Available: <https://arxiv.org/abs/2602.02710>
13. J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms." [Online]. Available: <https://arxiv.org/abs/1707.06347>
14. T. Degris, M. White, and R. S. Sutton, "Off-Policy Actor-Critic." [Online]. Available: <https://arxiv.org/abs/1205.4839>
15. J. Achiam, "Simplified PPO-Clip Objective." [Online]. Available: <https://drive.google.com/file/d/1PDzn9RPvaXjJFZkGeapMHbHGiWWW20Ey/view>
16. MiniMax et al., "MiniMax-M1: Scaling Test-Time Compute Efficiently with Lightning Attention." [Online]. Available: <https://arxiv.org/abs/2506.13585>
17. C. Zheng et al., "Group Sequence Policy Optimization." [Online]. Available: <https://arxiv.org/abs/2507.18071>